

MATHEUS JANCZKOWSKI

Introduction to python programming and self-study

A continuum mechanics perspective

Florianópolis, Brasil

2026

MATHEUS JANCZKOWSKI

Introduction to python programming and self-study

A continuum mechanics perspective

A didactic introduction to Python programming concepts
for self-study and early applications in continuum mechanics,
finite element methods, and scientific computing.

Florianópolis, Brasil

2026

Dados Internacionais de Catalogação na Publicação (CIP)
(Câmara Brasileira do Livro, SP, Brasil)

Janczkowski, Matheus

Introduction to Python programming and
self-study [livro eletrônico] : a continuum
mechanics perspective / Matheus Janczkowski. --
Florianópolis, SC : Ed. do Autor, 2026.

PDF

ISBN 978-65-02-02736-3

1. Mecânica do dano contínuo 2. Python
(Linguagem de programação para computadores)
I. Título.

26-349172.0

CDD-531

Índices para catálogo sistemático:

1. Mecânica : Física 531

Cibele Maria Dias - Bibliotecária - CRB-8/9427

© 2026 Matheus Janczkowski

All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without prior written permission of the author, except for brief quotations used in academic review, teaching, or scholarly citation.

First edition, 2026.

Published electronically at https://github.com/Matheus-Janczkowski/CompMechSelfStudy/tree/main/introduction_to_python_programming

Author: Matheus Janczkowski

Title: *Introduction to python programming and self-study*

Subtitle: *A continuum mechanics perspective*

ISBN: **978-65-02-02736-3**

DOI: **10.5281/zenodo.19361660**

Florianópolis, Brasil

This work is intended for educational, academic, and scientific use.

Preface

Continuum mechanics is a general-purpose tool to mathematically modeling physical phenomena around us. From a pulsating heart to the conformation of a glowing-red steel shaft, continuum mechanics is the broader framework to translate physical principles into differential equations. Due to the difficulty to finding analytical solutions to such differential equations, numerical methods have been proposed, with special attention to the finite element method (FEM). Numerical methods, such as the FEM, require computational effort and, thus, implementation using a programming language. In the past two decades, the *python* programming language has become widely used in the community of continuum mechanics, both for scientists and practitioners. There is a plethora of widely known and used modules and libraries in python, such as FEniCS for the FEM, GMSH for mesh generation, and TensorFlow for machine learning. This booklet presents basic python concepts and syntax intricacies to introduce the reader into the minimally necessary programming skills to translate continuum mechanics problems into python code and, from it, prospect results. The approach of this booklet is centered around brief conceptual presentation alongside programming examples mainly within FEM applications.

Florianópolis, 2026

Matheus Janczkowski

Contents

1	Introduction	7
2	How to learn how to code?	8
2.1	Logic	8
2.1.1	What is a variable?	9
2.1.2	Binary representation	9
2.1.3	Logic operations	10
2.1.4	Code fragmentation	11
2.2	Data and information	12
2.3	Understanding the problem	12
2.4	Design of a computer program	13
2.5	Testing	13
2.6	Integrating concepts	14
3	Brief note on programming paradigms	15
4	Elements of data	15
4.1	Index convention in python	16
4.2	Raw data	16
4.2.1	None (<i>NoneType</i>)	16
4.2.2	Binary statements (<i>bool</i>)	16
4.2.3	Numbers (<i>int</i> , <i>float</i>)	16
4.2.4	Strings (<i>str</i>)	18
4.3	Data structures	19
4.3.1	Lists (<i>list</i>)	19
4.3.2	Dictionaries (<i>dict</i>)	20
4.3.3	Numpy vectors, matrices and other arrays (<i>np.ndarray</i>)	22
4.3.4	Classes (<i>Class</i>)	23
5	Elements of logic	26
5.1	Indentation and division of code into blocks	26
5.2	Condition (<i>if</i> , <i>else</i> , <i>elif</i>)	27
5.2.1	Comparison operators	28
5.2.2	Relation operators	29
5.3	Repetition (<i>for</i> , <i>while</i>)	29
5.3.1	<i>for</i>	29
5.3.2	<i>while</i>	30
5.3.3	Regulators (<i>break</i> , <i>continue</i>)	31
5.4	Error management (<i>try-except</i>)	33

5.4.1	Reading and interpreting an error message	36
5.5	Basic compendium of algebraic and computation operations	38
5.6	Functions	38
5.6.1	Lambda function	42
5.7	Recursion	43
6	Assembly of all concepts, programming guidelines, and an example project	46
6.1	Code standards and aesthetics	47
6.2	Modularization	47
6.3	Example project	49
Appendix		55
Appendix A	Accessing the terminal and using the REPL	55
Appendix B	Test block with <code>__name__ == __main__</code>	56

1. Introduction

Programming has become an essential skill in engineering and applied mathematics, both in industry and in academia. Even if one uses commercial or stand-alone software, programming enables the individual to link and automate many tasks, which, in turn, makes their work more productive. If one heads towards building their own software, being able to customize is key. Consequently, even if open-source software and other out-of-the-shelf modules and libraries are to be used, programming is absolutely essential.

The python programming language has become over the past two decades the backbone of mainstream programming and machine learning. This programming language offers two main advantages: readable and simple syntax; connection with state-of-the-art software programmed in C language. This enables python to be easy to learn and connected with performative software for computationally demanding tasks. The ensemble of such characteristics makes python ideal for application in continuum mechanics and for the finite element method (FEM).

This booklet aims to introduce the reader to basic and essential python concepts. However, this text does not presume any programming knowledge of the reader; instead, it focuses on teaching a *programming philosophy*. Key concepts will be taught, alongside practical examples in python syntax. Fully copyable and reproducible code will be given, which the reader will be able to paste and test right on their own.

This work is structured in a first section dedicated to teaching core concepts of programming, such as logic, data, problem understanding, and testing; the second section introduces elements of data structures; the third section presents elements of logic. However, we stress that this booklet does not substitute for a formal text or course on programming. We also must notify beforehand that this work is highly biased towards continuum mechanics and, more specifically, FEM.

Additionally, this booklet is meant to be a quick guide and remembering device for those already familiar with programming (including the beginner who will be familiarized with programming after the completion of this text). In summary, this work has the objective to be quick and straight-to-the-point. Most important of all: **Have fun!**

This booklet is part of a series of text wrote for self-learning, i.e. students can read these texts and teach themselves how to do their own stuff. This booklet is mainly concerned with coding; for other quests, check:

1. Installing and software issues—<https://www.overleaf.com/read/wbxhncmtnmkm#0eb73b>.
2. L^AT_EX templates and utilities—<https://www.overleaf.com/read/sdrvfrpdjhft#66d6f9>.
3. Grante's suite for computational mechanics—<https://github.com/Matheus-Janczkowski/Grante>.

2. How to learn how to code?

When someone is faced with the challenge of needing to code but with no previous knowledge about whatsoever, the first question that comes to mind is: how am I supposed to learn how to code? The answer can be quite obscured by individual aesthetics and preferences. However, there is an underlying foundation to support any newcomer and help the reader developing their own style. Such foundation encompasses broader concepts, like logic and data structures; but also more abstract features, such as problem understanding and planning. This section aims to helping you to reach for each of the mentioned steps. Each of these steps will be discussed in their respective subsections.

2.1. Logic

Logic is a field of study of mathematics, it is concerned with segmenting or breaking down a *process* into a sequence of *arguments* or *steps*. What is a process? A process can be generally defined as a *task* of doing, building, or reaching for something. It must have a beginning and an end. It must have a single or a set of goals. Generally, people are interested in the goals themselves, but possess no clue on how to get there. The role of logic is precisely to trace the path from the beginning to the objective. This tracing is carried out, as already said, by setting a sequence of arguments or smaller units of reasoning, called steps.

An example of the application of logic outside of mathematics or computer science is *baking a cake*. One really wants to enjoy a succulent bite of sweet steamy cake, but does not have the slightest clue on how to make it. After doing some research, the gluttonous might find a *baking recipe*, which segments the *process of making the cake* into a sequence of steps:

1. gather the ingredients (flour, milk, eggs, sugar, fat, baking soda, and chocolate chips);
2. mix the dry ingredients thoroughly;
3. mix in the wet ingredients and reserve;
4. spread fat over the tray and pour in the dough mixture;
5. pre-heat the oven;
6. bake until the top is sufficiently solid.

It is clear that there is a process linking inputs (the raw ingredients) to an output (the baked cake). It is also clear that there is a sequence of steps, although this sequence is possibly not unique. A sequence of steps is called an *algorithm*.

Non-uniqueness of sequence can be seen in the cake example by swapping the pre-heating of the oven over to before the second step. Thus, the oven could be gathering temperature as the cook prepares the dough. This choice of algorithm could save time

compared to the algorithm enumerated above. Hence, the choices used to construct the algorithm affect the performance of the process, e.g. it can be finished faster or slower.

The reader is most definitely not interested in cooking when reading this text. Hence, we shall bridge those concepts to programming—the cake is something we desire, such as a finite element analysis or a trained neural network model, that is our output; the inputs are, alike the ingredients, necessary raw material for these outputs, such as material properties, meshes, boundary conditions, and neural network architectures; the process, i.e. the counterpart of the baking recipe, is the **computer program**, or code for short.

Logic was used in the cake example to break down the baking process into reasonably short and simple steps. Logic is used to define steps and construct algorithms for computer programs. For more detailed explanations on the field of logic, we refer to Abelson et al. [1]. It is most important to stress that logic is an abstract field and procedure. Therefore it is by no means tied to a programming language. As a consequence, you might have the same algorithm written differently in different programming languages, with the distinction being the syntax of the language.

Learning logic and how to break down a process into an algorithm is the most relevant skill for anyone interested in programming. Even at the dawn of artificial intelligence (AI), learning logic is paramount, since it underlies the building of larger, broader, and more complex computer programs. Additionally, it seems that learning logic makes anyone smarter and more rational.

The broader concept of logic in the programming universe (or in the universe of computer science (CS)) will be further subdivided into sections for more specialized care and detail.

2.1.1. *What is a variable?*

Computer science shares the concept of *variable* with mathematics, in the sense that it is defined similarly in both fields of study. Broadly speaking, variables are abstract entities that are *named*. They are abstract because they can assume different formats—numbers, matrices, vectors, strings of characters, and etc. Finally, variables are named, such as x , y , and A , for what they carry is linked to a name. To give an example, $x = 2$ means that the number 2 is carried by something that stores numbers and is associated with the name x .

Strictly from the computational standpoint, a variable is a storage *instance*, it is an object that stores a value assigned to it. The value corresponding to this object can be retrieved and used in different operations during the process. However, most importantly, the value within this object can *vary*, and a new value can be assigned to it.

2.1.2. *Binary representation*

The simplest information to be stored in a variable is *binary*. It can assume numerically only two possible values: 0 and 1. The 0 value is also perceived as a *false statement* (*False* in python syntax); whereas the 1 value is perceived as a *true statement* (*True* in

python syntax). This *type* of variable is called *boolean*, since it is the ground-matter for boolean algebra.

Binary representation ushered in the age of digital computing, because boolean data can be conveyed as the closing and opening of an electric circuit—1 if current flows through; 0 if no current flows. Such physical operations with electricity can also be used to represent and evaluate boolean operations. Many steps of the most basic tasks of computation, as addition and subtraction, can be broken down to boolean operations.

The choice of binary codification is due to reliability of assessing and measuring electrical current: the probability of measuring a low current as a higher current, and vice-versa, is much smaller than wrongly measuring numerous and less far apart thresholds of electrical current. This, however, is a trade-off of gain in reliability but loss with with representation capability, since more slots of memory (bits) are required to represent a number. That comparison is quite apparent with other representation approaches with more intermediate steps, such as decimals with 10 possible values.

Characters and symbols can also be represented using binary. However, they are first represented as a sequence of numbers and, then, these numbers are converted to binary, which become the binary codification of such characters.

Nevertheless, this whole commentary is a historical curiosity. Modern high-level programming languages possess higher levels of abstraction that allows us not to concern ourselves with individual operations within the electric circuitry of a computer; nor how data is codified and stored.

2.1.3. Logic operations

There are a few building blocks of logical statements; we opt to ensemble as such: conjunction (and); disjunction (or); negation (not); conditioning (if/else); repetition (for/do while). Some classical texts on programming do not include conditioning and repetition as logical operations, for they do not feature in boolean algebra. We, nonetheless, include them for simplicity, since they feature as building blocks to any code.

To illustrate the aforementioned operations, two generic variables x and y are used. Both of them store binary states only.

- Condition $\implies$ *if*: emits a statement, true or false, according to a question regarding the relation between two or among many variables.
- Repetition $\implies$ *for/do while*: executes a sequence of operations over one or more variables for a pre-fixed number of repetitions or indefinitely until a condition is met.
- Conjunction $\implies$ *and*:
 - **if** x is True **and** y is True, **then** x conjunction with y is True;
 - **if** x is True **and** y is False, **then** x conjunction with y is False;

- **if** x is False **and** y is True, **then** x conjunction with y is False;
- **if** x is False **and** y is False, **then** x conjunction with y is False.
- Disjunction $\implies$ *or*:
 - **if** x is True **or** y is True, **then** x disjunction with y is True;
 - **if** x is True **or** y is False, **then** x disjunction with y is True;
 - **if** x is False **or** y is True, **then** x disjunction with y is True;
 - **if** x is False **or** y is False, **then** x disjunction with y is False.
- Negation $\implies$ *not*:
 - **if** x is True, **then** negation of x is False;
 - **if** x is False, **then** negation of x is True.

The reader should notice that these operations underline basic thought structures and principles in our own natural languages, such as English, German, Latin, and Portuguese. We use those operations in our daily activities, as exemplified by the process of choosing between options. In programming languages, however, those statements are more concise, objective, and economical.

2.1.4. Code fragmentation

In biomechanics, one application of continuum mechanics, one notes that each organ performs a specific task—the heart pumps blood; the liver breaks down nutrients; the kidneys filter out byproducts. Yet, there is a central structure, the brain, that controls everything and makes sure that each organ plays its part in harmony with the whole ensemble. The same arrangement can be, and should be, used in computer programs.

Each step of an algorithm can be concealed to a **function** or a **method**. These objects receives input as arguments, perform some calculation, and return an output. One should observe the similarity to the concept of function as a mapping in mathematics. Moreover, function and method objects can be called to perform their tasks multiple times with the code for it being written only once.

Code fragmentation is a good practice since it reduces the size of the written code (redundancy is substituted by multiple calls of a single object). Fragmentation also improves readability and interpretability, for it is easier and faster to compile and read a single function instead of many repeated experts of code.

It is common practice to spread functions in different **routines**, i.e. python files. Each routine is specialized in doing a set of tasks that are somehow correlated to each other. For example, setting essential boundary conditions requires to find the nodes in the mesh where the boundary condition is applied and, then, enforcing the prescribed field value—finding the node and enforcing the prescribed value are different tasks, but both correlated

to each other and both belonging to the macro procedure of applying essential boundary conditions.

Each routine can be seen as a **module**, i.e. a bundle of code that can be loaded into other routines to provide to them useful functionalities. Usually, modules can be controlled by a central routine called *main*. The main routine, much like the brain, macro-manages the flow of information and the execution of other functions or of entire modules.

2.2. Data and information

Generically, a computer program is an algorithm or a set of algorithms that receives some information and returns some other information. Thereby, all the aforementioned blocks of logic are tools to process the **flow** of information through the code. Using this rationale, one might observe that the inputs constitute a **flux** into the code; whereas the outputs are a flux out from the code, they are the response of the program. **Data**, on the other hand, is the ensemble of stored information. Thus, input and output are also instances of data.

In nature, flows naturally evolve to a optimum configuration, i.e. the transported substance is often moved in the most efficient way possible, with the least possible consumption or dissipation of energy. The analogy is true in computer science—information must flow efficiently inside a code, otherwise it might spend too much computational resources or, worse, it might not even work at all. This phenomenon is also highly correlated to how the programmer encodes and structures the data within the computer program.

In summary, the reader, when challenged to write a computational code, must plan carefully how the data will be structured and how the corresponding information will flow inside and along the algorithm. This must be done in such manner that the idiosyncrasies of the programming language play to the advantage of the process. Therefore, besides planning and profound knowledge of logic, understanding the caveats of a programming language is quite useful. Surely, code performance and optimization is a concern to the more advanced enthusiast.

2.3. Understanding the problem

All that has been discussed so far is useful for all programming projects one might encounter. The realization and working of the project, however, will not come to fruition if the problem is not fully understood by the programmer. Understanding the problem is essential to building and stitching together the logic blocks, as well as to tailoring the data structures.

Note that the verb "to build", and its synonyms, has been used for many times up to this point. This is no coincidence: programming is a *design project* much like a design project in engineering. Designing something in engineering requires planning, organization, and study. The student should not start off programming; rather should the student start by studying the problem or procedure whose output is desired. The careful analysis of the

problem will enable the reader to identify the many steps to reach for the output. Fluency in programming, as is the case with any natural language, will naturally come with time and experience; exceptionally with larger diversity of problems. Thus, fluency must not be the immediate concern.

Many tools can be deployed to carry out such study and to organize ideas to tackling each problem. The choice is the product of the preference of the reader. Nonetheless, some tools will be delivered and discussed in the next subsection with the aim to helping designing a computer program.

2.4. Design of a computer program

Suppose that the reader has already assimilated the logic blocks and how information flows and is stored. Suppose also that the problem is completely understood and that the individual steps are cast. The reader might finally commence designing the computer program to solve the desired problem. Yet, the understanding is still abstract. A tool that is quite effective to translate the abstract to concrete code is *visual planning*, such as **flowcharts** and **pseudo-code**.

Flowcharts enable the programmer to write down each step as text blocks, which are connected by lines and other graphical signals. These graphical elements represent the flow of information and the sequence of steps. These features present a clear picture of how the algorithm must be organized. It is highly recommended that the student draws flowcharts. These flowcharts are invaluable tools to devising pseudo-codes.

Pseudo-code is the final structure of the planning process, and it is succeeded only by writing code using an actual programming language. Through pseudo-codes, the logic blocks can be tested, validated, and fully tailored. Pseudo-codes are free of vices related to syntax of a specific programming language, because they still hold a great deal of abstraction.

There is a misconception that writing flowcharts and pseudo-codes is the mark of someone at a basic level. For larger and more complex projects, often the product of collaboration between many programmers, flowcharts and pseudo-codes are invaluable for keeping track of the progress and for devising general and largely abstract structures.

2.5. Testing

A computer program that successfully *runs*, i.e. a program that successfully executes without any error, might conceal errors that will be apparent only when some conditions are present. These conditions might not be anticipated by the programmer, such that the error will appear much later. Those episodes are popularly known as **bugs**.

Many bugs appear when the programmer expects a set of operation conditions that are intrinsic to the modeled flow of information, whereas the flow actually happens differently. For this reason, each and every computer program must be tested for distinct operation scenarios. This means that the programmer must build a numerous ensemble of combinations of input data to test the routine to each one of them.

This phase is certainly the most frustrating of all. Testing and getting bad or even unexpected results surely discourage students to keep on learning and practicing coding. Nevertheless, testing is essential to guarantee, at least statistically, that a computer program is **robust**, i.e. that it is effective and flexible to different combinations of operation conditions. It is advisable to conceive and design every code from the beginning to be robust. This aim facilitates future alterations and expansions.

To test simple commands and simple behavior, one should do it in the *environment read-eval-print-loop* (REPL). Instructions to access and manage the REPL are shown in Appendix A. One option of framework to testing a single module in python, or a more complex strip of code, is presented in details in Appendix B.

2.6. Integrating concepts

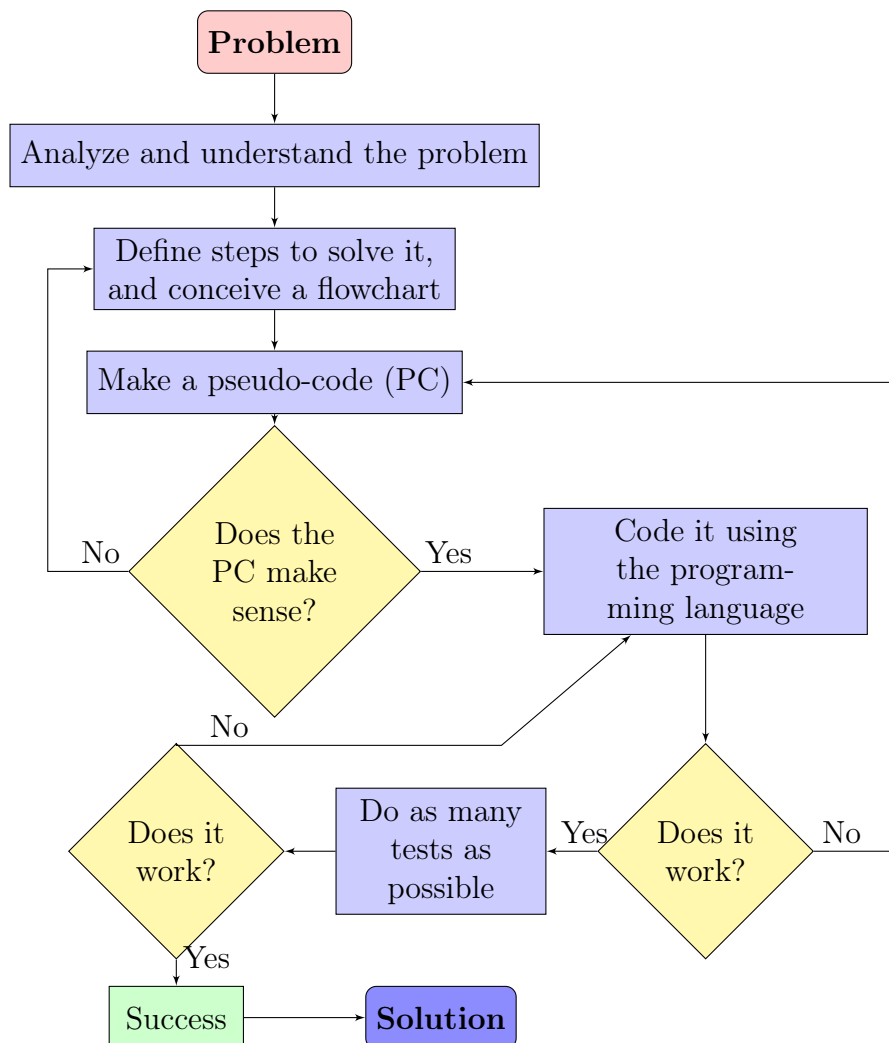


Figure 1: Flowchart to illustrate how to program.

The totality of concepts presented to this point can be read as part of a coarse handbook on how to program. It is coarse because there is no hint about how to actually write code in any specific programming language. However, the objective of teaching a

philosophy on how to program was reached; which is, as a matter of fact, lacking in many programming courses.

One of the discussed themes was the organization of the steps to solving a problem or process using graphical elements. The flowchart shown in Fig. 1 demonstrates this theme metalinguistically: the steps for programming, which include devising a flowchart, are drawn and connected in that flowchart using balloons and logic ligatures.

3. Brief note on programming paradigms

The reader might have already heard about the concept of *programming paradigms*, or most certainly will do in the future. Programming paradigms are styles or philosophies of programming. Briefly, if coding is meant to be a set of instructions on *how to do something*, this is an *imperative paradigm*; if coding is about declaring *what is to be done*, it is called *declarative paradigm*. We focus on imperative paradigms, as it suits best our intentions and own style in this work.

Imperative paradigm can be further segmented into *procedural paradigm*, where each step, or procedure, is set in an organized sequence, and into *object-oriented paradigm*, where each step can be interpreted as an *object*. The concept of objects broadly means a set of data and a set of operations contained within. We will see that python is essentially a object-oriented programming language. One consequence of that is classes and functions, for example, being interpreted as objects, much like data structures, such as lists and dictionaries, or even raw data, as exemplified by numbers and strings.

For the curious reader, we refer to Van Roy and Haridi [2] for more details about programming paradigms.

4. Elements of data

As hinted before, data is stored information that is readily available for processing. There is generically two types of data: *raw data*, which is exemplified by numbers, characters, strings, boolean variables; and *data structures*, that are more complex structures to store raw data in some order or using some kind of heuristic, such as lists, dictionaries, and classes. In python, interestingly, both raw data and data structures can be seen as objects.

The two categories of data will each have a corresponding subsection in the following text. In each subsection, there will be subsubsections dedicated to each **type** of data. The name of the class or type of the data will be presented in the title of each subsubsection to familiarize the reader with python types. To assess the type of an object, the reader shall use the function *type()*.

Code examples will be shown throughout, and they can be executed in a python file (.py), or they can be directly executed in REPL. If you opt to use the REPL for repeating the examples, check Appendix A for instructions on how to access it.

4.1. Index convention in python

There are types of data, raw or structured, that can have elements of themselves accessed individually at a time, which is called *indexing*. Indexing happens by providing a number that tells the order of the required element in the data instance. That order prescribes a *counter*, which is an integer number that must initiate at a first value. There are two main conventions for the first ever counter, the programming languages that start their counter at: 0, such as python and C; and those at 1, like Julia and Matlab. In python, in particular, indexing is carried out by giving the counter in between brackets.

The last element is given by the index -1. Ranges of indices are given by the first desired index and the last desired index plus one separated by ":"— as an example, from the second to the third indices, 1:3. Examples will be dully shown for each data type.

4.2. Raw data

4.2.1. None (NoneType)

None is the absence of a value, is a completely empty variable. It is the default value returned by a function that does not have a return statement. It is particularly useful for asserting if a variable has been given as argument, or if it has the proper value, for the cost to store that variable is negligible. See the code example to define a None variable and to verify its type,

```
1 variable = None
2 type(variable)
```

Code 1: The result of type(variable) is 'NoneType'.

4.2.2. Binary statements (bool)

Binary variables can be either True or False. Hence, they are the simplest type of data. Binary variables are commonly used to control process and pass information along the code to inform future steps whether something happened or not. Binary variables used to inform the code about the state of something are called **flags**, for they signal some condition. Check Code 2 for syntax example.

```
1 true_variable = True
2 false_variable = False
3 type(true_variable)
4 type(false_variable)
```

Code 2: Boolean variables and their type. The result of type(variable) is 'bool'.

4.2.3. Numbers (int, float)

As discussed previously, since the dawn of the era of digital computing, electric circuits have been used to convey information in binary codification, see Subsubsec. 2.1.2.

There are a limited number of bits to represent a single number, which are segmented into groups of 8 bits, called *bytes*. Bytes are directly linked to the architecture of the computer and of its processors. From this, different number representation schemes stemmed, such as **int32**, **int64**, **float32**, and **float64**; the number in these names tell the number of bits for storing the said numbers in binary.

We focus on two families of number representation in memory: *int* for integers; and *float* for floating point numbers, e.g. real numbers. *How* those numbers are codified and stored is not the focus of this booklet; the curious reader might check any book on numerical analysis, such as [3, 4]. The two cited books also address a fundamental problem in floating point arithmetic: the propagation of errors due to the error in representing real numbers in a finitely precise representation scheme.

A brief commentary on a key trade-off in choosing number formats: for integer numbers, int64 is expected to represent a wider range of numbers than int32; for floating point numbers, float64 is expected to represent numbers with higher precision in a wider range, too. However, the 64-bit format consumes twice as much memory as the 32-bit format. Hence, 32-bit format is faster than the 64-bit format and this is the reason why many machine learning frameworks are optimized for 32-bit format, such as TensorFlow in python.

Indices for indexing lists and arrays must **always** be integers. For calculation, it is better practice to write numbers explicitly as float (3.0 instead of 3 for example). The latter is meant to avoid conversion from integer to float under the hood, which might make the code slower. One should use integers in operations only if the result is meant to be an integer, such as for indices.

Float variables can be converted to integers. Strings with numbers as characters can be parsed to integer and floating-point numbers. Functions native to python are used for those purposes. These functions will be listed in Subsec. 5.5.

Example code for defining and using numbers is provided in Code 3. Note that some special numbers, such as π , are available in python libraries, most notably the numpy library. Code 3 also provides the natively available algorithmic operations with numbers. For other operations, such as square root, logarithms, and trigonometric functions, one should use a library such as numpy, TensorFlow, or UFL (FEniCS). Scientific notation is given by the coefficient value followed by E and the exponent, $3.14 \times 10^6 = 3.14E6$, see line 20 of Code 3.

```
1 integer = 3
2 real = 3.14
3 import numpy as np
4 float_pi = np.pi
5 # Verifies the type of each variable
6 type(integer)
7 type(real)
8 type(float_pi)
```

```

9 # Addition
10 integer+real
11 # Subtraction
12 integer-real
13 # Multiplication
14 integer*real
15 # Division
16 integer/real
17 # Power
18 integer**real
19 # Scientific notation
20 real = 3.14E6

```

Code 3: Numbers and their type. The result of `type(integer)` is 'int'; the result of `type(real)` and of `type(float(pi))` are 'float'.

4.2.4. Strings (*str*)

Strings are known as such for they are *strings of characters*. Characters are letters, symbols, and numbers in between quotation marks, such as "a" or 'a'. They are enormously versatile and are used quite often to make code flow of information readable. Strings can be used as: names for files; paths for directories; keys for dictionaries; features to be printed on the screen; variables to forward some information that must be readable, even if codified.

Strings could be interpreted as a data structure, because they are the assembly of another smaller units, the characters. Much like lists (as we will see), they can be appended (more characters can be added), and they can be indexed (a character or a set of them can be selected given a numerical index of its position in the string). However, strings are not often interpreted as data structures for they ensemble only a type of raw data: characters; thus, they lack generality and high level operations usually present in data structures.

```

1 character = "%\"
2 string = "Hello world! It's 32C"
3 # Indexing
4 string[0]
5 string[1:5]
6 # Takes out the last character
7 string = string[0:len(string)-1]
8 # Adds another character to the end
9 string += "F"
10 # Adds another string or character to a middle position
11 string = string[0:6]+"freezing "+string[6:len(string)]
12 # Transforms something else into a string
13 something_else = str(3.0)
14 # Type
15 type(string)
16 type(character)

```

Code 4: Strings and their type. The results of both `type(character)` and `type(string)` are `'str'`.

Code 4 shows how a character and a string can be defined. Then, at line 3, it demonstrates how the string can be indexed; line 4 returns `'H'`, whereas line 5 returns `'ello'`. At line 7 of Code 4, the variable is shortened to `"Hello world! It's 32"`. Then, at line 9, the variable becomes `"Hello world! It's 32F"`. Finally, at line 11, a whole string is added to a middle position, whose result is `"Hello freezing world! It's 32F"`. The function `len()` returns the number of characters in string; more details about these functions will be given later. Line 13 shows how other variables can be converted into a string: the number 3.0 is converted to `'3.0'`.

4.3. Data structures

Data structures are higher-level options for storing data and, possibly, other features such as functions. They follow some kind of order and, therefore, are called structures. Data structures usually have built-in features that allow for fast data recovery and construction. This section will be divided into the main data structures used in python.

4.3.1. Lists (*list*)

Lists are *arrays* of virtually **any data**. Lists in python are sets of *elements* separated by commas and arranged in brackets. Lists can be **nested** entities, i.e. they can have lists as elements as well, so much so that there can be lists inside lists. Lists that are nested as elements of another list are usually called as **sublists**.

Lists are widely used objects for storing data that naturally appear in some kind of order. The order, thus, is reflected in their indices. Examples of syntax, definition, and usage are shown in Code 5. One interesting feature of python is *list syntax*, which enables the creation of a list from elements directly as the result of a for-loop inside brackets (check line 16 of Code 5).

```
1 list_variable = [1.0, 2, "Hello", ["W", [np.pi, 0, "Pi"], 2.0]]
2 # Verifies the type
3 type(list_variable)
4 # List indexing
5 list_variable[0:3]
6 list_variable[3]
7 list_variable[3][0]
8 list_variable[3][1][1]
9 # Indexing a string character inside a list
10 list_variable[3][1][2][1]
11 # Appending an element to a list
12 list_variable.append(3.0)
13 # Appending an element to a nested list
14 list_variable[3].append(3.0)
```

```

15 # Appending multiple elements to a list but not creating a nested sublist
16 list_variable.extend([4,5,6])
17 # Creating a list using "list syntax"
18 list_syntax = [2*i for i in range(4)]

```

Code 5: Lists and their operations. The result of `type(list variable)` is 'list'.

Line 1 in Code 5 shows how a list with data and with nested lists can be defined simply by using the elements separated by commas inside brackets. Line 5 illustrates how a range of indices can be retrieved from the original list, in this case from the first to the third indices, whose result is `[1.0, 2, 'Hello']`. A single element can be retrieved with a single index, as does line 6, whose result is the nested list `['W', [3.141592653589793, 0, 'Pi'], 2.0]`. The first element of that nested list can be recovered with the indexing `[3][0]`, as presented at line 7, with the result 'W'. Finally, the second element of the nested list inside the first nested list can be retrieved with the indexing `[3][1][1]`, see line 8, which gives 0.

Note that the third element of the list `[np.pi, 0, "Pi"]`, nested in the list nested in the list variable, is a string, "Pi". The second character, "i", can be recovered simply by appending the index of the said string to the set of indices of the list: `[3][1][2][1]`, whose result is indeed 'i'.

An element can be appended to a list by using the attribute `.append(element to be appended)`, such as in line 12 of Code 5, whose result is `[1.0, 2, 'Hello', ['W', [3.141592653589793, 0, 'Pi'], 2.0], 3.0]`. Similarly, appending can be carried out to a nested list. If the number 3.0 is to be appended to the sublist nested in the fourth element of the list variable, one should do as in line 14 of Code 5, which results in `[1.0, 2, 'Hello', ['W', [3.141592653589793, 0, 'Pi'], 2.0, 3.0], 3.0]`.

If multiple elements are to be appended to a list as elements instead of another nested sublist, one shall use the attribute `.extend(list of elements to be appended)`. An example is presented in line 16 of Code 5, whose result is `[1.0, 2, 'Hello', ['W', [3.141592653589793, 0, 'Pi'], 2.0, 3.0], 3.0, 4, 5, 6]`.

An interesting feature of python is constructing lists with something called *list syntax*. As exemplified by line 18 in Code 5, each element of the list, that is generated by a for-loop, is succeeded by the corresponding loop. The result for this particular example is `[0, 2, 4, 6]`. The range function returns a set of the integers from 0 to the provided integer argument minus 1, 3 in this case (as $4-1=3$).

4.3.2. Dictionaries (*dict*)

Often, data must be *labeled*, which means that data come in pairs of label and value. Data can be virtually always stored in a list, but integer indices lose readability, they do not readily mean anything, especially for the user. To illustrate the problem, take the following example: suppose there is a domain with two regions, say *nucleus* and *outer ring*; one must store information for each region and recover it independently later. The first approach

would be a list,

```
1 information = [nucleus_information, outer_information]
```

Hence, for the code to get the information for each region, it would need to index the information list with `information[0]` and `information[1]`. However, the indices 0 and 1 do not readily mean "nucleus" and "outer ring" for the user, which might lead to confusion and error. Thus, it would be safer if each information variable could be assigned to a key with the name of the region.

The desired behavior lastly described is exactly how a dictionary works. Code 6 shows a code example for this particular application of two regions in a domain with different properties. The properties are stored in a list corresponding each to a domain region, see lines 2 and 3 in Code 6. Then, a dictionary is created directly by inserting in between brackets the pairs of *key* and *value*. The key-value pairs are written as the key followed by `:` and, then, the value. The pairs are separated by commas. See line 5 in Code 6 to understand how the explicit setting of key-value pairs is carried out in python syntax. For this example, the keys are the strings with the names of the domain regions; and the values are the lists with the corresponding properties.

```
1 # Initializes the values as a list of two properties
2 nucleus_information = [0.49, 1E6]
3 outer_information = [0.4, 15E6]
4 # Direct definition of a dictionary
5 dictionary = {"nucleus": nucleus_information, "outer ring": outer_information}
6 # Iterative definition of a dictionary
7 dictionary = dict()
8 dictionary["nucleus"] = nucleus_information
9 dictionary["outer ring"] = outer_information
10 # Accessing information using a key
11 dictionary["nucleus"]
12 # Modifying the element of a list that is a value in a dictionary
13 dictionary["outer ring"][1] = 11E6
14 # Deleting a key-value pair
15 dictionary.pop("nucleus")
16 # Other types of keys
17 dictionary[10.0] = 30.0
18 dictionary[(1,2)] = "same value for two keys compressed in a tuple key"
19 # Type of a dictionary
20 type(dictionary)
```

Code 6: Dictionaries and their operations. The result of `type(dictionary)` is 'dict'.

Nonetheless, a dictionary can be built first empty, as in line 7 of Code 6, using the command `dict()`. Afterwards, the dictionary will be populated each key-value pair iteratively, such as in lines 8 and 9 for the nucleus and the outer ring, respectively.

To access the value corresponding to a key, one must simply use the syntax `dictio-`

nary[key], as exemplified in line 11 of Code 6, whose result is exactly [0.49, 1E6]. Values of a dictionary can be altered *in place*. If one wants to modify an element of a list that is stored in a dictionary for instance, the list will be accessed using the key and, then, the element will be retrieved using common list indexing, as shown in line 13 of Code 6. A key, nevertheless, cannot be changed. If one provides a new key expecting for it to be changed, another key-value pair will be created. A key-value pair can be deleted using the attribute .pop(key to be deleted), just like in line 15 of Code 6.

Other types of keys besides strings can be used, namely: integer and float numbers; and tuples. Tuples behave like lists, but are immutable. Tuples are particularly useful as keys when the same value is intended to be paired with two different keys, but it is not desirable to spend memory twice with the same information. One must use tuples for this application for python does not allow lists as keys. Check lines 17 and 18 of Code 6 for illustrations of these concepts.

4.3.3. Numpy vectors, matrices and other arrays (*np.ndarray*)

There are no native vectors or matrices in python as there are in Julia or Matlab. Those kinds of data structures, in their mathematical sense, are available in python libraries, such as numpy, SciPy, and TensorFlow. The implementation in each one of these libraries has a different focus—numpy is general purpose; SciPy has good handling of sparse matrices; TensorFlow has exceptional handling of arrays focusing on parallelization and performance, especially within the context of machine learning. We, however, will focus on numpy, for it is the most generic and widely used format. Although, one should keep in mind that much of the syntax of numpy is shared with the other two libraries.

```
1 # Imports numpy library if it has not yet been imported
2 import numpy as np
3 # Gets a numpy vector and a matrix from their respective lists
4 vector = np.array([1.0, 2.0, 3.0])
5 matrix = np.array([[5.0, -1.0, 0.0], [-1.0, 6.0, 3.0], [0.0, 3.0, 8.0]])
6 # Multiplication of matrix by vector
7 np.matmul(matrix, vector)
8 # Multiplication and sum of matrices
9 np.matmul(matrix, (3.0*matrix)+matrix)
10 # Norm of vector
11 np.linalg.norm(vector)
12 # Solution of linear system of algebraic equations using LU decomposition
13 np.linalg.solve(matrix, vector)
14 # Indexing of vector and matrix elements
15 vector[1]
16 matrix[1,0]
17 # Generating vectors and matrices of zeros
18 null_vector = np.zeros(3)
19 null_matrix = np.zeros((3,3))
20 # Populating elements using indices and ranges of indices
```

```

21 null_vector[1] = -2.0
22 null_matrix[1,0] = -1.0
23 null_matrix[2,:] = [0.0, 3.0, 8.0]
24 # Verifies their types
25 type(vector)
26 type(matrix)

```

Code 7: Numpy arrays and their operations. The results of `type(vector)` and of `type(matrix)` are `'np.ndarray'`.

Code 7 shows how vectors and matrices can be constructed in numpy as well as some important operations with them. Line 4 illustrates how a numpy vector can be constructed from a list of elements using the `array` function. Conversely, line 5 shows how a matrix can be defined using the same function but a list of sublists, where each sublist is a row of the final matrix.

Multiplication of a matrix to a vector and of a matrix to another matrix is carried out using the `matmul` function, which assures matrix multiplication convention. Both uses are presented in lines 7 and 9 of Code 7. However, multiplication of those arrays by a constant and addition among them are carried out using the same operation syntax `*` and `+`, respectively.

Operations common to linear algebra, such as evaluating the Euclidean norm and solving a linear system of algebraic equations using the LU decomposition, are available in the submodule `linalg`. Check lines 11 and 13 of Code 7.

Vectors and matrices can be indexed as lists, but the indexing syntax is different—in numpy, all indices are stacked inside a single pair of brackets and they are all separated by commas, if there is more than one. See line 15, that yields 2.0, and line 16, that yields -1.0, in Code 7.

Vector and matrices can be initialized as null arrays using function `zeros`. If a null vector is to be created, the argument will be the number of elements in the vector, see line 18 of Code 7. If a null matrix is to be created, the argument will be a tuple of the numbers of elements in each dimension of the array, see line 19 as example for a 3×3 matrix. After, the elements that are not null can be populated using indexing, such as exemplified in lines 21 and 22 of Code 7. Additionally, whole rows or columns can be populated at once, if `:` is specified as the index for the dimension that will be entirely populated, see line 23. The value of `null_matrix` after line 23 is `[[0.0, 0.0, 0.0], [-1.0, 0.0, 0.0], [0.0, 3.0, 8.0]]`.

4.3.4. Classes (Class)

Classes are high level data structures that act as a fancy hybrid of a tool box with a cupboard. Classes can store data, as any of the raw data and any of the data structures listed before. Besides, classes can store functions, which are called **methods** because they are linked to a class. For these reasons, classes are extremely versatile objects and enable complex coding patterns to emerge quite easily. Moreover, classes enables code to be easily

modified and updated later, for they are not hard-coded as functions are. Classes are the epitome of object-oriented programming.

Classes have some built-in methods, most famous are two: `__init__`, that is for initializing a class and storing the necessary data; `__call__`, a method that is executed whenever the class is called as a function. The `__init__` method is present at the definition of any class and, thus, will be our focus. The `__init__` method is the first method executed when a class is initialized, and it is the method responsible for creating the object conventionally called *self*, which stores all data and all other objects inherent to the corresponding class.

Any time a class is initialized, an **instance** of this class is created. Using an analogy, the class can be interpreted as the pattern of a cupboard, e.g. it must have two shelves, three drawers, and one door. Whereas the instance is a particular exemplar of that cupboard pattern, such as an exemplar filled with white china, a jar of milk, and some sacks of flour. Similarly, one could fill another instance of the same cupboard pattern with tin cups, a bottle of soda, and some sacks of sugar. The job of filling the storage pattern with one particular choice of data, and, thus, of creating the instance, is the role of the `__init__` method.

Each unit of data and each object that belongs to a class are attached to it using *attributes*. Attributes have names as any variable and, consequently, they work more or less like keys in a dictionary. Data attributes are usually populated into the self object when the `__init__` method is executed. The methods defined in the class are automatically pulled to the self object when the instance is initialized.

Even though the cupboard analogy seems straightforward, it is hard to grasp **where** and **when** to use classes in a day-to-day programming situation. The first answer is simple data storage, because one can store labeled data in it, where a value is associated with an attribute (much like a value is associated with a key in a dictionary). The second answer is creating objects with other high-level objects as attributes of itself, such as functions (called methods in the context of classes).

Reading the last paragraph, the reader might argue that dictionaries also store labeled data and can carry high-level objects, such as lists, dictionaries, functions, and even class instances. Hence, the immediately following question is: why should one use a class instead of a simple dictionary? Classes have a structure and an architecture set in its pattern when the pattern was first defined. Classes can be constructed from a parent class, i.e. forcing the user to keep some obligatory methods and attributes. Classes are a great way to enforce pattern, structure, and inheritance.

To illustrate the aforementioned concepts and ideas, let us appreciate an example. Suppose we want to evaluate the function $f(x) = a \exp(bx)$. The parameters a and b are real constants that can vary, and, for each combination of them, one gets a different model. Here is the perfect example of an application of something that follows a blueprint—as

symbolically defined by the function—but has complete freedom to vary some parameters or attributes. Thus, it is straightforward to propose a class with a and b as data attributes and a method to evaluate the exponential function.

Implementation-wise, the blueprint for this application is set in the class definition `ExponentialFunction`, see lines 5 to 18 in Code 8. The two data attributes, a and b , are stored into the self object in the `__init__` method, which is executed immediately when the class is instantiated in line 22 of Code 8. The final touch of the blueprint is to have a method defined for evaluating the exponential function $f(x)$, which is precisely performed by the method `evaluate`. Note that the `evaluate` method receives as argument the self object, for it contains the data attributes.

At line 22 of Code 8, metaphorically speaking, the cupboard pattern is filled with china and flour: the attributes values are fed into the initialization of the instance. The resulting object, `exponential_instance`, is the class instance, and it has all the attributes of the pattern. Consequently, `exponential_instance` can be used to evaluate the exponential function for that particular set of parameters at a value of x , see line 25 of Code 8.

```
1 import numpy as np
2
3 # Defines a class to evaluate an exponential function with given parameters (the
4 # cupboard pattern)
5 class ExponentialFunction:
6
7     # Defines the __init__ function to receive two constants as arguments
8     def __init__(self, a, b):
9
10         # Stores the given arguments as attributes of the self object, which is a
11         # keyword for all that is stored inside the class
12         self.a = a
13         self.b = b
14
15     # Defines a function to evaluate the exponential function at x
16     def evaluate(self, x):
17
18         return self.a*np.exp(self.b*x)
19
20 # Instantiates the class with a particular set of parameters (the cupboard
21 # exemplar)
22 exponential_instance = ExponentialFunction(2.0, 3.0)
23
24 # Uses the evaluate method to evaluate the exponential function at x=1.5
25 exponential_instance.evaluate(1.5)
26
27 # Evaluates a particular attribute
28 exponential_instance.a
29
```

```

30 # Changes a particular attribute
31 exponential_instance.b = 2.5
32
33 # Verifies if a class instance has an attribute with a given name
34 hasattr(exponential_instance, "a")
35
36 hasattr(exponential_instance, "evaluate")
37
38 hasattr(exponential_instance, "c")

```

Code 8: Basic code example for creating, instantiating, and using a class.

The instance `exponential_instance` can be used to assess the used values for the parameters. One can check the value of the a parameter, for example, by checking the `a` attribute, as in line 28. Conversely, one can change an attribute by assigning a new value to it, see line 31 of Code 8. Additionally, the existence of attributes in a class instance can be verified using the `hasattr` function and the name of the desired attribute as a string. See, for example, lines 34 and 36 of Code 8; as `a` and `evaluate` are indeed attributes of `ExponentialFunction` class, `hasattr` will return `True` in both cases. Nevertheless, at line 38, the result will be `False`, since that class does not possess an attribute called `c`.

Note that classes are usually named with capital initials and no underline division, this is a pythonic convention; not a rule. Many aspects of this section, and specially of Code 8, will be discussed later, since they are essentials of logic not of data. Examples of those aspects are functions, methods, and indentation. Those features reassure the image of classes being data structures that live in a higher level of abstraction. Nonetheless, those aspects will be remembered in the upcoming sections.

5. Elements of logic

In Sec. 2, it was stated that programming consists of designing how information should be coded into a flow of data and, then, how this flow is trimmed, transformed, and processed. We have viewed to this point, namely in Sec. 4, different ways to convey and to structure data. We presented a glimpse on logic operations in Subsubsec. 2.1.3. This section is devoted to translating those logic operations into python syntax.

5.1. Indentation and division of code into blocks

Every code is divided in blocks. Each block performs a particular task. Some blocks verify conditions; other perform some process repetitively in a loop; some blocks call a function; and etc. It is possible, and often desirable, to have some blocks confined inside other block. Thus, it is absolutely necessary to separate the realm of one block from the others, so that the interpreter knows what does what when the code is executed.

Each programming language developed its own way of fencing code blocks: Java and C use curly brackets (`{}`); Julia and Fortran use `end` as key to finish a block; Python uses

indentation. Indentation is a set number of whitespaces (conventionally 4 spaces from one block to another) used in front of each line. When the code is seen on an editor, an indented block appears to be visually *inside* another block. This phenomenon is clear in Code 8—see that the `def __init__` and `def evaluate` bits are dislocated to the right with respect to class `ExponentialFunction`, which means these two statements are inside `ExponentialFunction` block. In the other hand, lines 10 to 13 are inside the `def __init__` block; and line 18 is inside the `def evaluate` block. In practice, when something is inside something else, the former is executed only when the latter is executed or already initialized.

5.2. Condition (*if*, *else*, *elif*)

A condition begins with a simple question: if (something) is True, then do something in the block of code bellow. See lines 2 to 4 in Code 9 for example, if the variable `flag` is True, the `if` flag bit will be satisfied and the interpreter will execute what is inside that statement block, which is to print "The flag is True". Three cases might appear after: the statement is complete and nothing more must be tested; there is only a single more case to evaluate that is the exact opposite of the statement previously assessed; there are multiple cases to be verified besides the previous statement. The first case will not be further discussed, because it is already finished.

The second case is shown in lines 5 and 6 of Code 9—there is only one other possibility, the flag being False, thus, an `else` statement is used. If the flag is indeed False, the code block inside the `if` statement in line 3 will not be executed and the interpreter will skip directly to the code block inside the `else` statement in line 5. In this case, the code will execute the printing of "The flag is False".

The third case is when more than one condition must be evaluated after the first statement, then an `elif` statement must be used. Between lines 8 and 16 of Code 9, an example of situation with three conditions is presented. The `exponential_instance` variable must be tested for having the attribute 'a'; even if it does not have that attribute, it can still be an instance of the class `ExponentialFunction`, which is tested by the `isinstance` function and returns True if the first argument is indeed an instance of the second argument. Finally, if `exponential_instance` does not have an attribute 'a' and if it is not an instance of the class `ExponentialFunction`, the code inside the `else` statement is executed.

The two examples shown before feature boolean flags or functions that return booleans, such as `hasattr` and `isinstance` functions. Lines 18 to 30 in Code 9 illustrate examples of conditions using comparisons, such as equality, inequality, and belonging. If the variable name is the string 'square root', the block in line 22 will be executed. Otherwise, if the variable is None, the block in line 24 will be executed. If both conditions before fail and the variable name is an element in the list of prohibited functions, the block in line 26 will be run. Again, if all previous conditions are False and the name is different to "exponential", block in line 28 is activated. Finally, if everything before is False, the interpreter falls into

line 30.

```
1 # Boolean conditions
2 flag = True
3 if flag:
4     print("The flag is True")
5 else:
6     print("The flag is False")
7
8 # Conditions with functions that return boolean
9 if hasattr(exponential_instance, "a"):
10     print("The instance has an attribute named 'a'")
11 elif isinstance(exponential_instance, ExponentialFunction):
12     print("The instance is an instance of the class ExponentialFunction, but does"+
13         " not have the attribute 'a'")
14 else:
15     print("The instance is not an instance of the class ExponentialFunction and "+
16         "does not have and attribute named 'a'")
17
18 # Conditions with comparison
19 name = "exponential"
20 prohibited_functions = ["logarithm", "sine"]
21 if name=="square root":
22     print("Square root is asked")
23 elif name is None:
24     print("name is None")
25 elif name in prohibited_functions:
26     print(str(name)+" is asked, but it is not an allowed function")
27 elif name!="exponential":
28     print("Something other than exponential is asked")
29 else:
30     print(str(name)+" is asked")
```

Code 9: Code examples of condition statements.

5.2.1. Comparison operators

To assess comparisons, as in the example featured between lines 18 and 30 in Code 9, relation and comparison operators must be defined. These operators test the veracity of the comparison and return True or False accordingly. A set of operators (**O**) will be listed as follows, and the format *if X O Z* is shared by all of them.

1. Equality: **O** $\rightarrow$ ==
2. Inequality: **O** $\rightarrow$!=
3. Greater than or equal to: **O** $\rightarrow$ >=

4. Lesser than or equal to: **O** $\rightarrow$ `<=`
5. Greater than: **O** $\rightarrow$ `>`
6. Lesser than: **O** $\rightarrow$ `<`

5.2.2. Relation operators

In the following list, X and Y are conditions built upon the operators defined in Subsubsec. 5.2.1.

1. Conjunction (and): X *and* Y
2. Disjunction (or): X *or* Y
3. Negation (not): *not* Y

5.3. Repetition (*for*, *while*)

A repetition loop consists of a block of code that is repetitively executed in two possible scenarios: for a pre-set amount of times (**for**); or indefinitely until a *stop condition* is met (**while**).

5.3.1. *for*

For-loops have the same blueprint of: **for**; the iterated object; **in** a set, list, or set of pairs; indentation and the block of code to be executed at each iteration of the loop. See Code 10 to note this pattern.

The simplest loop is constructed using the *range* function, which builds a set of counters starting at 0 and, increasing at a step of 1, up to the given argument minus 1. For example: `range(5)={0, 1, 2, 3, 4}`. This idea can be complexified using the *linspace* function of numpy, which yields a set of *n* evenly spaced counters from the initial value to the final value. See lines 1 to 12 of Code 10 for these two examples.

If the for-loop is to iterate over the elements of a list, there are two options: the first is to use the range function and feed as argument the length of the list (`len`) and, then, index each element (see lines 15 to 17); or, more efficient and more pythonic, iterate over the elements directly (see lines 20 to 21).

When iterations over a dictionary are required, there are three options: iterating over the keys, lines 25 to 26; looping over the values, lines 29 to 30; or, iterating over both keys and values simultaneously, lines 33 to 34. The choice between the aforementioned options is due to the required data—if one requires keys and values inside the block of code, the third option is the best; if one requires keys or values only, the previous options fit better.

```
1 # Generic for-loop for n iterations
2 n = 5
3 for i in range(n):
```

```

4     print("Executes iteration number "+str(i))
5
6     # Generic for-loop in a range with initial and final values, and the given number
7     # of iterations
8     initial = 1.0
9     final = 2.0
10    n = 5
11    for i in np.linspace(initial, final, n):
12        print("Executes counter "+str(i))
13
14    # For-loop through the elements of a list using the length of the list
15    list_variable = ["a", 1.0, 2]
16    for i in range(len(list_variable)):
17        print(list_variable[i])
18
19    # For-loop through the elements of a list
20    for element in list_variable:
21        print(element)
22
23    # For-loop through the keys of a dictionary
24    dictionary = {1: "one", 2: "two"}
25    for key in dictionary.keys():
26        print(key)
27
28    # For-loop through the values of a dictionary
29    for value in dictionary.values():
30        print(value)
31
32    # For-loop through the keys and values of a dictionary
33    for key, value in dictionary.items():
34        print("Key "+str(key)+" corresponds to value "+str(value))

```

Code 10: For-loop. Examples with ranges, lists, and dictionaries.

5.3.2. *while*

While loops iterate over a code block indefinitely until a stopping criterion is met. This stopping condition can be performed using a control variable and a comparison statement, much like the example between lines 2 and 6 of Code 11. Or the stopping condition can be evaluated using a specially crafted boolean flag, that must be updated inside the repeated code block, as exemplified between lines 9 and 16 in Code 11.

When someone writes while loops, special attention and care must be devoted to it, for stopping conditions can be wrongly defined and, as a consequence, the code will not get out of the while loop. Ultimately, this faulty behavior will hinder performance dramatically, and crash the routine entirely. For this reason, for-loops are usually preferred whenever possible.

```

1 # Using a control variable as stopping criterion
2 t = 0
3 while t<5:
4     print(t)
5     # Update of the control variable
6     t += 1
7
8 # Using a flag as stopping criterion
9 flag = True
10 while flag:
11     print(t)
12     # Update of an internal variable
13     t = t*10
14     # Update of the stopping criterion flag
15     if t>4000:
16         flag = False

```

Code 11: While loop. Generic example to show the breaking condition.

5.3.3. Regulators (*break*, *continue*)

Often a loop (for or while) is defined over a maximum number of iterations, but nothing is expected to be done in a particular iteration, or nothing is required to be done from that iteration to the next ones. In these cases, it is necessary to stop the loop to avoid idle computational effort. To this end, *regulators* are used, such as *break* and *continue*. There is the *pass* as well, but this one will be discussed in the context of error management.

The *continue* regulator is used when that particular iteration is idle, has nothing to be done, but future iterations surely will have tasks to complete. The *break* regulator, on the other hand, is used when from that iteration to the next ones, the task is fully complete and nothing is left to be carried out. Both regulators are, thus, found in condition blocks (if, elif, or else) inside a repetition loop. It is important to highlight that the regulators affect only the repetition loop they are immediately inserted in. Hence, if two or more repetition loops are nested into one another, only the immediate loop is broken or skipped by the selected regulators.

Tridiagonal matrices are a good example to show how regulators can be used in practice, and how they behave. Tridiagonal matrices are a special kind of sparse matrix that appear in many scientific applications, such as spline interpolation and finite element simulations of 2D trusses with linear finite elements. Such matrices are null matrices except for the main diagonal and the two diagonals adjacent to each side of the main one. Tridiagonal matrices allow us to use the numpy knowledge gained in Subsubsec. 4.3.3 and show the use of nested for-loops to iterate over row and column indices.

To make this example more interesting, suppose that some rows of the matrix are nonzero only at the main diagonal. One of the possible algorithms is presented in Code 12;

although it is clearly and deliberately not the most efficient algorithm for this task. We, indeed, challenge the reader to devise a better algorithm and to stretch their programming skills looking for less iterations and less condition statements.

Definition of variables, the setting of nested for-loops, and the condition statements will not be discussed from now on. We move straight to the main logic behind Code 12. Inside the two loops for row and column indices to assemble the matrix elements one by one, there is a first condition statement (see lines 19 to 33) to test if the row number is one of the rows selected to have nonzero elements only at the main diagonal. Then, between lines 35 and 38, it is verified if the column number is within the bandwidth of the tridiagonal matrix. Finally, from line 40 to line 44, a condition statement is used to verify if the column number is beyond, to the right, the band, thus, being null by definition.

```
1 import numpy as np
2
3 # Sets the parameters for the width of the bands
4 inner_band_width = 2
5
6 # Sets some rows to have nonzero elements on the main diagonal
7 diagonal_rows = [3, 5, 7]
8
9 # Sets the matrix dimension and initializes it as a null array
10 n = 14
11 matrix = np.zeros((n,n))
12
13 # Iterates over the rows
14 for i in range(n):
15
16     # Iterates over the columns
17     for j in range(n):
18
19         # Verifies if this diagonal is one with only a diagonal entry
20         if i in diagonal_rows:
21
22             # Verifies if the column number is equal to the row number
23             if i==j:
24
25                 matrix[i,j] = 2.0
26
27                 # And breaks the loop over the columns
28                 break
29
30             # Otherwise, skips to the next column
31             else:
32
33                 continue
34
```

```

35     # Verifies if the column number is within the central band
36     elif j>(i-inner_band_width) and j<(i+inner_band_width):
37
38         matrix[i,j] += 1.0
39
40     # Verifies if the column number is greater than the band
41     elif j>=(i+inner_band_width):
42
43         # Breaks this loop and skips to the next row
44         break

```

Code 12: Example of break and continue regulators used to construct a tridiagonal matrix.

Inside the verification if the row number is one of the rows to have a single diagonal element, there are two other conditions. The first condition verifies if the column number is equal to the row number, what positions the corresponding element on the main diagonal. The second condition reasons otherwise. Inside the first condition (see lines 22 to 28), the diagonal element is populated and, then, the loop over the column indices is stopped using the break regulator, for there is no reason to continue iterating in j if all the elements are zero from this point on. Inside the second condition, the situation is the opposite—the column counter has not reached the row number yet, thus needs to continue updating, but the remainder of the code inside the for-loop can be skipped.

The verification condition of the band elements from line 35 to line 38 is straightforward and requires no explanation. The condition from line 40 to line 44 is again a block to avoid idle iteration. If the column index surpassed the bandwidth, all the elements are null and there is no reason to keep on looping over j . Consequently, a break regulator is used to stop the loop over the column indices, which ultimately skips to the next iteration in the row indices.

It is important to highlight that the most commonly used regulator is break, since the behavior of the continue regulator can often be mimicked using if-elif statements. For example, there is no reason to use the block between lines 30 and 33, because all the code blocks within the j loop are contained inside condition statements chained to each other using if-elif structures. Hence, when the interpreter runs into the verification block between lines 19 and 33 and the condition in line 23 is not met, it automatically would skip to the next iteration in j if there were no block in lines 31 to 33.

This behavior, however, is possible if and only if all condition statements are chained together using if-elif-else; if there were any if block after line 44, the continue regulator in line 33 would be indeed necessary.

5.4. Error management (*try-except*)

Error is when something unexpected happens and the interpreter is not capable of resolving some operation given some data as input, which leads to a crash. Errors can

be divided in two main categories: bad programming and bad usage. The first one is the easiest to solve and becomes ever rarer as the programmer gains experience and skill. Bad programming can be solved during the phase of writing code itself, performing tests and validating performance. Bad usage, on the other hand, is the worst nightmare of any programmer, because it requires a great deal of empathy of whom is writing code to foresee how the user will actually use the code.

Good documentation of the code and how it is meant to be used is a must-have prevention feature. Nevertheless, it is not sufficient because many users simply ignore. For this reason, the programmer must armor the code against bad usage and fill it with contraptions to stop bad and unexpected behavior and, most importantly, notify it in the clearest possible manner. Fortunately, programming languages possess the exact built-in tool for this task, that is the try-except blocks.

Blocks of try-except syntax have a pattern of two blocks: the trial, where the commonly expected before is tried to be executed as normal; the exception, which the interpreter executes if something failed during trial. There are three variants—the except block just try something else or simply ignore the error and the program does not crash; the except block conceals a **raise** command to stop the code from running ahead and a custom error message can be inserted; the except block has the raise command and an alias to show the original error message. All variants are shown in Code 13.

```
1 # Tries something and moves on
2 list_variable = [1.0, 2.0, 3.0]
3 index = 3
4
5 try:
6
7     list_variable[index]
8
9 except:
10
11     print("Does something else and does not stop or crash the code")
12
13 # Tries something and stops with a custom message
14 try:
15
16     list_variable[index]
17
18 except:
19
20     raise IndexError("The index "+str(index)+" is not a valid index")
21
22 # Tries something and stops with the original error message
23 try:
24
```

```

25     list_variable[index]
26
27 except Exception as e:
28
29     raise IndexError("The index "+str(index)+" is not valid. The original"+
30     " error message is: '"+str(e)+"'")

```

Code 13: Examples of the three variants of try-except using a list indexing error as theme, since errors with lists are usually due to wrong indexing.

Code 13 shows a single type of error, that is wrongly indexing a list. The `list_variable` has 3 elements and the index 3 was used, but it is not allowed, since the maximum index in python convention is 2 for this case. This error was chosen as example due to its simplicity, for it is expected that most errors related to lists are due to bad indexing. One shall note this fact is explicit in lines 20 and 29, where the raise command throws an **IndexError**, i.e. an indexing error.

Nevertheless, try-except blocks are not strictly necessary, since if-else blocks could be used instead, just as in Code 14. The same problem and the same logic is used here, but the programmer must understand the problem first, since condition statements do not automatically identify errors as try-except blocks do. Due to this lack of automation, using condition blocks to prevent errors and raise error messages is a most important programming exercise. This exercise builds the ability of the programmer of understanding the code flow in details as well as the skill of thinking as the user.

```

1 # Uses condition block to raise an error
2 list_variable = [1.0, 2.0, 3.0]
3 index = 3
4
5 if not isinstance(list_variable, list):
6
7     raise TypeError("'" + str(list_variable) + "' is not a list, thus cannot be "+
8     "indexed")
9
10 elif index >= len(list_variable):
11
12     raise IndexError("The index "+str(index)+" is not valid for indexing list "+
13     str(list_variable))
14
15 list_variable[index]

```

Code 14: Example with list indexing error that can be handled and raised within a condition block.

In Code 14, two distinct kinds of error are foreseen: a type error, where the `list_variable` might not actually be a list; an index error, where the index might be off of the true limits of a valid index, just like in the previous example. This new setting sheds a glimpse into the possibility of different errors, other than index errors. More kinds of error will be discussed

in Subsubsec. 5.4.1.

Observe that the `raise` command was used as before, in the `try-except` blocks. This command will throw a custom error message in the terminal or prompt just like a exception would. Using the `raise` command is quite advisable, since it enables the proposition of custom error messages, which are much more meaningful and helpful than the generic built-in error messages of python.

Finally, note that it was not necessary to conceal the normal and expected bit of code in a `try` block; rather it was left as usual after the condition blocks. This is a clever design, because the error checking is completely carried out in a prior and single phase. Thus, the order is different: in `try-except` blocks, the normal and expected bit of code is tried first, then the exception is handled if it appears; with condition blocks, the order is reversed and the checking must be done first. Again, however, we highlight that the programmer must know the expected errors to use the condition blocks strategy; which is, on the other hand, a superb exercise for the reader.

5.4.1. *Reading and interpreting an error message*

As stated before, different errors can happen. We briefly enumerate some of the most common ones:

1. **AttributeError**—when a name of an attribute of a class is actually not an attribute of it. For example, if `exponential_function.c` would trigger an `AttributeError` in Code 8, since `'c'` is not an attribute of `exponential_function`.
2. **FileNotFoundError**—when a file is asked to be loaded from a path or complete path and it is not found.
3. **ImportError**—when a module is tried to be imported but it does not exist in the current instalation of python or it is not available in the current path of the interpreter.
4. **KeyError**—when a key is used to access a value in a dictionary but it is not actually a key of that dictionary. For example, if one tried to get `dictionary["inner ring"]` in Code 6, this error would appear.
5. **NameError**—when a variable is used in a bit of code, but it does not exist in the local scope of the code where it is supposed to be used. As an example, if some operation was carried out in line 15 of Code 12 with the counter `j`, but it is not defined outside of its own `for`-loop.
6. **TypeError**—when some operation is carried out between two variables of types that cannot be operated together. It is also thrown when a wrong number of arguments is given as input to a function. As an example, if one tries to construct a matrix in `numpy` without given the number of rows and of columns in a tuple, like if it were `null_matrix = np.zeros(3,3)` in Code 7.

7. **ValueError**—when some value is of the correct type but wrongly given as the input to a function. For example, if one tries to convert a string to an integer, but it is not possible, such as `int(string)` in Code 4.

For a complete list of built-in exceptions, check <https://docs.python.org/3/library/exceptions.html>.

When facing an error, even as the code writer themselves, the reader must know how to interpret the **traceback** message. The traceback is a full list of errors and how they occurred through the whole code that appears in the terminal when an error happens. Suppose that a particular program is composed of numerous python files, each with distinct functions that are executed inside other functions of the same program. In summary, it is a completely modularized computer program (as every one should be). The traceback message shows the complete list of functions that failed due to a single error.

Even though functions will be carefully explained in the next sections, they are useful to show the concept of traceback. Feel free to read about functions and, then, come back here. In Code 15, three functions are defined to perform simple mathematical operations. However, one calls another; the only self-sufficient function is `function_1`. Thus, if `function_3` is called with a non-numeric argument, the error will appear first in `function_1` and, then, be propagated back to `function_2` and, finally, `function_3`. This behavior is exactly what happens at line 19 of Code 15, where a string is fed to be used as argument to mathematic expressions.

```
1 import numpy as np
2
3 # Defines a bunch of functions
4
5 def function_1(x):
6
7     return x**2
8
9 def function_2(x):
10
11     return function_1(x)+3.0
12
13 def function_3(x):
14
15     return np.exp(function_2(x))
16
17 # Forces an error trying to execute function 3 with a string
18
19 function_3("two")
```

Code 15: Example of composed functions and the use of it which will throw an error.

The traceback message is designed to show the last operation at the end. The most recent operation, likely where the error occurred, is shown at the bottom of the message. This choice of presentation helps identify the problem much faster, since the origin of the issue is readily above the terminal bar. Then, working from the last call to each one of the previous calls, the whole traceback is shown, just as exemplified in Code 16.

One can see the true diagnostic of the problem at the bottom line, line 13 of Code 16, which tells that a string is not supported for operand `**`. Then, one observes that this error occurred in line 7 of the code shown in Code 15, in `function_1`. But, `function_1` was called in line 11 of Code 15 in `function_2`, exactly as tells line 7 of Code 16. Conversely, the same occurred for the calling of `function_2` in line 15 of Code 15 in `function_3`, as shows line 4 of Code 16. Finally, the initial call is presented at line 2 of Code 16, which tells that the last line to fail was precisely the calling of `function_3` with "two" as argument, in line 19 of Code 15.

```

1 Traceback (most recent call last):
2   File "/home/janczkowski/teste.py", line 19, in <module>
3     function_3("two")
4   File "/home/janczkowski/teste.py", line 15, in function_3
5     return np.exp(function_2(x))
6         ~~~~~
7   File "/home/janczkowski/teste.py", line 11, in function_2
8     return function_1(x)+3.0
9         ~~~~~
10  File "/home/janczkowski/teste.py", line 7, in function_1
11    return x**2
12        ~~~~
13 TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'int'

```

Code 16: The resulting traceback of example in Code 15.

5.5. Basic compendium of algebraic and computation operations

Table 1, Table 2, and Table 3 present a small list of some basic functions and operations that are quite useful in a daily basis. Care is taken to present a brief explanation and the corresponding syntax. If any library is necessary to use any of the functions, they will be explicit in the syntax and in the caption.

5.6. Functions

During the past sections, in many examples and in many excerpts of text, the name *function* was cited. In Tab. 1, Tab. 2, and Tab. 3, a plethora of functions was presented, as means for many different tasks. However, the nature of such objects was not discussed. This is exactly the objective of this subsection.

Functions in programming languages, much like their counterparts in mathematics, are a mapping from a space of inputs V (or **arguments** in the pythonic jargon) to another

Operation	Syntax	Operation	Syntax
Addition	+	Subtraction	-
Multiplication	*	Division	/
Power	**	Square root	<i>np.sqrt()</i>
Exponentiation	<i>np.exp()</i>	Natural logarithm	<i>np.log()</i>
Logarithm of base a of x	<i>np.emath.logn(a, x)</i>	Logarithm of base 10	<i>np.log10()</i>
Sine	<i>np.sin()</i>	Cossine	<i>np.cos()</i>
Tangent	<i>np.tan()</i>	Absolute value	<i>abs()</i>
Divisor of a by b	$a//b$	Remainder of division of a by b	$a\%b$
Real part of a	<i>a.real</i>	Complex part of a	<i>a.imag</i>
Integer factorial	<i>math.factorial()</i>		

Table 1: Basic mathematical operations. *np* stands for numpy, thus the numpy library must be imported. Conversely, *math* stands for the math library.

Operation	Syntax	Operation	Syntax
Determinant	<i>np.linalg.det()</i>	Trace	<i>np.trace()</i>
Vector of eigenvalues	<i>np.linalg.eigvals()</i>	Matrix of eigenvectors	<i>np.linalg.eig()[1]</i>
Matrix inverse	<i>np.linalg.inv()</i>	Pseudo matrix inverse	<i>np.linalg.pinv()</i>
Solution of $\mathbf{Ax} = \mathbf{b}$	<i>np.linalg.solve(A, b)</i>	Cholesky decomposition	<i>np.linalg.cholesky()</i>
Euclidean norm	<i>np.linalg.norm()</i>	p -norm de x	<i>np.linalg.norm(x, ord = p)</i>
Maximum value of X	<i>np.max(X)</i>	Minimum value of X	<i>np.min(X)</i>
Mean	<i>np.mean()</i>	Standard deviation	<i>np.std()</i>
Variance	<i>np.var()</i>	Median	<i>np.median()</i>

Table 2: Basic algebraic and statistics operations using the numpy library.

space of outputs W . Thus, a function f is mathematically written as $f : V \rightarrow W$. This means that a function receives a set of quantities, the arguments, processes them, and returns something useful elsewhere.

Functions in python can be divided in three options related to their arguments: no arguments; **positional arguments** only; positional arguments and/or **keyword arguments**. When a function does not have any argument, it already has within itself everything necessary to perform its task. Positional arguments are necessary arguments, the user must provide them and in the order they were defined in the function. Keyword arguments are optional arguments which already have default values to it, so the user is not required to supply them.

```

1 # Defines the basic blueprint of a function in python without arguments
2 def no_arguments():

```

Operation	Syntax
Print	<code>print()</code>
Round x to n decimal digits	<code>round(x, n)</code>
Convert to string	<code>str()</code>
String concatenation of x and y	<code>$x + y$</code>
Convert to integer	<code>int()</code>
Convert to float	<code>float()</code>
Range starting at 0 and up to $x - 1$	<code>range(x)</code>
Range with $start$, $stop$, and $length$	<code>np.linspace($start$, $stop$, $length$)</code>
Appends X to y	<code>$y.append(X)$</code>
Length of list or array	<code>len()</code>
Verifies if X is an instance of y	<code>isinstance(X, y)</code>
Verifies if X is an attribute of y	<code>hasattr(y, X)</code>

Table 3: Basic computational operations.

```

3
4     return 3.0
5
6 # Defines the basic blueprint of a function in python with positional arguments
7 # only
8 def positional_arguments(x, y):
9
10     return x+y
11
12 # Defines the basic blueprint of a function in python with positional and keyword
13 # arguments
14 def generic_function(x, y, z, a=1.0, b=2.0, verbose=True):
15
16     # Performs some calculation with the given arguments
17     result = x+(y*a)+(z*b)
18
19     # Prints something if the verbose flag is True
20     if verbose:
21
22         print("The result of 'generic_function' is "+str(result))
23
24     # Returns the result
25     return result
26
27 # Example of calling the function with no arguments and the function with posi-
28 # tional arguments only
29 no_arguments()
30 positional_arguments(3.0, 4.0)
31
32 # Example of calling the generic function without giving the keyword arguments
33 generic_function(3.0, 4.0, 5.0)

```

```

34
35 # Example of calling the generic function with keyword arguments
36 generic_function(3.0, 4.0, 5.0, b=3.0, a=1.5)

```

Code 17: Blueprint of a function object in python

Those three options of functions are illustrated in Code 17. A function with no argument is shown between lines 1 and 4; a function with positional arguments only is defined between lines 6 and 10. Finally, a generic function, with positional and keyword arguments, is defined from line 12 to line 25. The execution of the two first kinds of functions is shown from line 27 to line 30 of Code 17.

It is important to highlight that order matters for positional arguments. For example, in line 30, *x* assumes the value of 3.0 and *y* assumes the value of 4.0. In line 33, *z* assumes the value of 5.0, as it does in line 36. With keyword arguments, order is not important since the name of the argument (key) is there to relate the value to the variable. For that reason, *b=3.0* can come before *a=1.5* in line 36. In Code 17, line 36 also illustrates that not all keyword arguments must be provided at once. In this line, *verbose* takes its default value, *True*.

In addition to the arguments, we should stress the role of the **return** statement in the blueprint of a function; see lines 4, 10, and 25 of Code 17. This command effectively assembles the variables to be output by the function. If no return statement is written, the function will return a *None* instance. The output of the function is a tuple if more variables are after the return command (such as *return a, b*), the order of this tuple is the same as the order of the laid variables.

Argument-less functions are often used to manage processes, such as the usually named *main* functions that kick-start any major computer program. Functions with positional arguments only are fairly common, as they emulate mathematical operations. Functions with keyword arguments are really useful to enable default values for users. Besides, keyword arguments are essential when altering a function that had already been written and is used by another section of the code. One does not want to alter the calling and execution of that function in the rest of the code, thus keyword arguments are defined and a default value is set, which will be used in the rest of the code.

Functions are the most efficient way to segment code. Functions encapsulate a set of instructions that can be performed for an indefinite number of times without the need of having to write the same lines of code over and over again. Consequently, functions are an indispensable tool against code redundancy, and for improving code readability and interpretability.

Functions are also particularly useful for modularization and construction of the *inner map* of the code. This inner map of the code is used later for tools concerned with parallelization and with automatic differentiation. Thereby, modularizing code with functions is paramount for improving code performance.

5.6.1. Lambda function

One cannot use the term **lambda** as a name for anything in python, the reason lambda being a *reserved name* in that programming language. Reserved names are names of objects that are globally (enveloping an entire code) identified and intrinsic to python, thus the user is not allowed to alter them. One such example is the lambda function.

The lambda function is an interesting and extremely useful object in python, since it enables the programmer to create simple functions in just a few lines of code. When the lambda function is used, the traditional *def name(arguments):* syntax is not necessary. In contrast, one has only to write *lambda argument_1, argument_2, ..., argument_n: expression*. This syntax can be better demonstrated by line 5 of Code 18. Another interesting feature is the possibility of assigning the resulting object to a variable, such as *exponential_function* in Code 18.

```
1 import numpy as np
2
3 # Definition of an exponential function f(x, a, b) = a*exp(b*x) using the lambda
4 # function
5 exponential_function = lambda x, a, b: a*np.exp(b*x)
6
7 # Usage of a lambda function as any other
8 print("The result is "+str(exponential_function(1.0, 2.0, 3.0)))
```

Code 18: Example of a lambda function in python.

However, one particular important use of lambda functions is to create **drivers**. Drivers are functions that return the output of another function while fixing some of their arguments. This, again, is much easier to understand with an example—suppose the reader is interested in numerically evaluating the derivative of the exponential function defined in Code 18 with respect to the x variable using central finite differences (CFD). CFD requires just the x variable to be perturbed, hence it is straightforward to create another function that is function of x only. This function is precisely a driver.

A code proposition for the example above is presented in Code 19. The original exponential function with all three arguments (x , and the two coefficients a and b) is defined as a traditional python function between lines 3 and 6. Then, values for the coefficients are proposed in lines 9 and 10 of Code 19. These values are used to create a driver for the exponential function using the lambda command in line 12.

A function was defined from line 14 to line 24 to perform the whole CFD calculation. Note, nevertheless, that function *CFD_derivative* must receive a function f as argument that is a function of a single variable; and this is the justification for the driver in Code 19. The result is finally presented in line 28. This example shows the applicability of lambda functions and drivers in a real context of computational mechanics.

```
1 import numpy as np
```

```

2
3 # Defines the exponential function with all arguments
4 def exponential_function(x, a, b):
5
6     return a*np.exp(b*x)
7
8 # Defines the fixed values of coefficients a and b, then, creates the driver
9 fixed_a = 2.0
10 fixed_b = 3.0
11
12 driver = lambda x: exponential_function(x, fixed_a, fixed_b)
13
14 # Defines a function to evaluate CFD
15 def CFD_derivative(f, x, h):
16
17     # Evaluates the function ahead of the x point
18     f_ahead = f(x+h)
19
20     # Evaluates the function abaft of the x point
21     f_abaft = f(x-h)
22
23     # Returns the derivative approximation
24     return (f_ahead-f_abaft)/(2*h)
25
26 # Evaluates the derivative of the exponential function at x=1.0 passing the driver
27 # as f. A step of 1E-5 is used
28 print("The derivative at x=1.0 is "+str(CFD_derivative(driver, 1.0, 1E-5)))

```

Code 19: Use of lambda function to define drivers. Example with numerically calculated derivatives using central finite differences.

5.7. Recursion

Suppose that the reader should write a computer program to search for a particular file in a set of directories. Given the logic subsidies of Sec. 5, the reader would certainly propose to get a list of directories and use a for-loop to search for each directory in the list. Suppose, now, that, in each directory, there are other nested directories, so much so that the first list of directories are called *parents* and the nested ones are called *children*. Very well, the reader would follow its intuition and propose yet another for-loop, nested in the first one, to iterate over the child directories.

Let yet another level of complexity be added: what if each parent directory has a different number of child directories in it? Still, nested for-loops would work, since the number of iterations of the nested loops could be iteratively accessed by the parent for-loop. Another name for this is a *tree structure*, since branches (parents) sprout and, from each branch, other branches (children) sprout.

Using for-loops would work indefinitely if there were always parallel branches, i.e. no parent directory could stop having children if other parent directories at the same branch *level* still had children directories. This thought exercise is left to the reader to fully grasp and, perhaps, to code by themselves.

As hinted by the last paragraph, using for-loops for the task of reaching each directory in a tree structure is limited. In fact, it breaks down if some branch does not have other child branches while others at the same level do. There is a tool to tackle this problem, which is called **recursion**, and it also comes from mathematics. Curiously, recursion is not taught in programming courses in engineering faculties. This, surely, is very strange, due to the wide applicability of recursion.

Another immediate example of recursion application is recovering elements and search in lists. This application is particularly notable in python, since lists in python can become quite *unruly*, in the sense that nesting sublists can be mirrored onto a tree structure with no particular order.

We delivered an example where traditional logic statements fail to solve the problem; then, we presented recursion as a tool to tackle such problem. However, we did not define nor presented what is recursion. Recursion is formerly defined as a function that calls itself during some step of its own set of sets. There are two classical and simple examples of recursive functions: the generator function of the Fibonacci sequence, $f(n)$; and the function to evaluate the factorial of an integer number, $g(n)$,

$$\text{Fibonacci: } f(n) = f(n-1) + f(n-2); \quad f(0) = f(1) = 1, \quad n \in \{2, 3, \dots\}, \quad (1)$$

$$\text{Factorial: } g(n) = ng(n-1); \quad g(0) = 1, \quad n \in \{1, 2, \dots\}. \quad (2)$$

In each one of the recursive functions in Eq. (1), it is clear how recursive functions work—they need to call a **previous instance** of itself to calculate the current state.

In addition to calling itself, there is another defining feature of a recursive function: the **base state**. The base state is the most basic configuration of any recursive function. From that state, the recursive function cannot call itself once again. When a recursive function reaches the base state, the opposite happens and the flow of information reverses—the function is evaluated as any other in the base state and its output is fed to the last call of itself.

One should now note that this pattern of transmitting the output to the corresponding call of the function will propagate until the original call is reached. To exemplify this workflow, we should check the factorial function. Suppose, we would like to evaluate the

factorial of 5,

$$\begin{aligned}g(5) &= 5g(4), \\g(4) &= 4g(3), \\g(3) &= 3g(2), \\g(2) &= 2g(1).\end{aligned}\tag{3}$$

$g(1)$ is precisely the base state of the factorial function, and it is defined as $g(1) = 1$. Therefore, with the base state, we can proceed to reverse the flow direction and trace the function calls back,

$$\begin{aligned}g(2) &= 2g(1) = 2, \\g(3) &= 3g(2) = 6, \\g(4) &= 4g(3) = 24, \\g(5) &= 5g(4) = 120.\end{aligned}\tag{4}$$

The basic structure of recursion is set, we shall now present examples of code of recursive functions in python. Code 20 presents a function to evaluate the factorial of an integer, see lines 1 to 10, and a function to print on the terminal each element of a list, see lines 15 to 30.

The base state of the factorial function is tested in line 5. If the base state is found, the function returns the value 1. Otherwise, the function must call itself once again following the recursive relation, as is done in line 10.

A function to print each element of a list must loop over the elements of the list received as argument, as carried out in line 19 of Code 20. For each element that is read in the for-loop, it must be tested if the element is yet another list, which is done in line 32. If the element is indeed a list, the function will call itself again and feed as argument that particular element (see line 25). Otherwise, if the element is not a list, the base state is reached and the **desired action** can be executed, which is printing the read object.

```
1 # Defines a recursive function to calculate the factorial of an integer
2 def factorial(x):
3
4     # If it is equal to 1, returns the base state
5     if x==1:
6
7         return 1
8
9     # Otherwise, returns the recursive relation
10    return x*factorial(x-1)
11
```

```

12 # Tests with the factorial of 5
13 print("The factorial of 5 is "+str(factorial(5)))
14
15 # Defines a recursion function to print each element of a list
16 def list_printer(list_object):
17
18     # Iterates over the elements
19     for i in range(len(list_object)):
20
21         # Verifies if this element is a list
22         if isinstance(list_object[i], list):
23
24             # Calls the function again
25             list_printer(list_object[i])
26
27         # Otherwise, it is the base state and prints it
28         else:
29
30             print("The element is "+str(list_object[i]))
31
32 # Tests with a list with no proper order
33 list_printer([1.0, 2.0, [3.0, [4.0, 5.0], [6.0, 7.0], 8.0]])

```

Code 20: Two examples of recursion: a function to evaluate the factorial of an integer; a function to print each element of a list.

As Code 20 is executed on terminal, the output is presented on Code 21. The printed result is accurate and follows the pattern we expect. The function devised to print objects in a list hints the logic behind many recursion applications, such as file and directory searching and mapping.

```

1 The factorial of 5 is 120
2 The element is 1.0
3 The element is 2.0
4 The element is 3.0
5 The element is 4.0
6 The element is 5.0
7 The element is 6.0
8 The element is 7.0
9 The element is 8.0

```

Code 21: Result on the terminal of the code wrote on Code 20.

6. Assembly of all concepts, programming guidelines, and an example project

Concepts have been presented independently so far. Everything has been discussed in their tiny little boxes. From this point on, we shall glue all the previous tools into one body of knowledge, focusing on real programming skills.

6.1. Code standards and aesthetics

Programming is a team endeavor. Even if you write code alone, you do it, in fact, as a team with your self of the future. For this very reason, writing code in a standardized and clear way, thinking as a team, is the safest alternative for your coworkers and yourself to understand that piece of code now, and in the future. Additionally, code good practices help avoiding bugs and errors; or, at least, diagnosing them more easily.

What are those code practices though? There follows a list,

1. **Choose names that are meaningful:** name variables, functions, files, and other objects with names that convey the purpose of the corresponding object. Do not use symbols, blank spaces, nor non-ASCII characters for naming.
2. **Code width:** do not write too many characters in a single line of code. We prefer to use a maximum of 72 characters (personal choice). This way, code can fit entirely on a screen and there will be no need for using side-moving bars.
3. **Commenting:** almost every line should be commented. Comments must tell the story of the logic the programmer used to write the corresponding code. Comments help logic identification and, thus, make much easier to add functionalities and fix bugs later.
4. **Create your own rules and standards:** and stick to them. Either for your coworkers or for yourself, it is much easier to understand, read, and work on predictable code. Standards offer safe amounts of predictability.
5. **Line spacing:** skip lines whenever possible. Avoid writing cluttered code that is difficult to read.
6. **Modularization:** code must be broken down into functions and classes. These objects must be placed within files where they naturally belong. Create one file for each broader category of tasks in a project. If a particular category of tasks is organized well enough, it might be reused in future projects.
7. **Write in English:** code wrote in English can be posted on online forums, such as StackOverflow. These online forums gather programmers and enthusiasts from all over the world. They will gladly look at anyone's code and offer help only if it is written in English.

6.2. Modularization

In Subsec. 5.6, it was hinted that functions are useful to avoid code redundancy. Functions and classes can be reused in different **places** of the code. Place in a code can refer not only to a particular line, but also to whole files. Files, such as multiple python

files, are wholesome alternatives to keeping code short and objective. A single file can host different functions and classes programmed to help with a task. Hence, that file becomes specialized in this task.

Segmenting a program in different files precisely defines the process of modularization. Python, conveniently, perceives each file as a **module**, much like widely used libraries, such as numpy. Thus, for a file to have access to objects defined in another file, this must be imported in that. Suppose there are two .py files **in the same directory**, `tangent_matrix.py` and `gauss_quadrature.py`. Suppose as well that `gauss_quadrature.py` has a function called `evaluate_gauss_points`, which must be used in `tangent_matrix.py`. To use this function inside `tangent_matrix.py`, `gauss_quadrature.py` must be imported in the header of that file, as exemplified in Code 22.

```
1 # Routine to store methods to evaluate the tangent matrix
2
3 # Import the gauss_quadrature module
4 import gauss_quadrature as gq
5
6 # To use the function to evaluate the Gauss points, one must only do
7 gq.evaluate_gauss_points()
```

Code 22: Example of importing a python file as a module.

In the example of Code 22, only a single function should be imported. Thus, if `gauss_quadrature.py` were a file full of numerous objects, it would not make any sense to import such a heavy file to use just a single function. In such cases, we can import just what we want, as shown in Code 23.

```
1 # Routine to store methods to evaluate the tangent matrix
2
3 # Import the evaluate_gauss_points function from the gauss_quadrature module
4 from gauss_quadrature import evaluate_gauss_points
5
6 # To use the function to evaluate the Gauss points, one call it as normal, with no
7 # reference to the module
8 evaluate_gauss_points()
```

Code 23: Example of importing a single function from a python file.

Recursive importing, however, is not allowed. Recursive importing happens when a file imports another file as a module, but this file imports that file as a module, or another still that eventually imports the first file. This constraint must be in mind to actually design how the different tasks will be divided among modules. Since tasks are inevitably correlated by some sort.

Usually, there is a file called *main.py* that controls all processes. The main file will not necessarily import all modules, but will call the necessary functions to do all the work that is intended to be done.

6.3. Example project

This booklet is complete, at least in its aim, by presenting an example project. The code examples presented to this point are too small and too specific. There must be a **project** to gather as many concepts as possible and put them to use. This project cannot be too long (in terms of lines of code), and cannot be too complicated; but cannot be too short either.

There are a plethora of examples of real-world projects in continuum mechanics and in finite element analysis. When continuum mechanics and the finite element method are put together, the result is computational mechanics. We must stress to the reader that the computational mechanics example ahead will be conceptually easy and simple. We refer the work of Holzapfel [5] for a background on continuum mechanics, and the works of Simo and Hughes [6], Wriggers [7] for details on the finite element method.

Nonetheless, a brief review of continuum mechanics and stress. Stress tensors are mappings from the normal vector of a slice through a point to the corresponding traction vector (forces per area) that pushes that slice facet. The traction vector is defined to keep balance of forces and moments in the remaining fraction of the body. Thus, stress tensors arise as mathematical objects to keep balance of momenta (and the laws of Newton in continua). A particular stress tensor is the first Piola-Kirchhoff tensor, which relates tractions to normal vectors in an undeformed configuration of the body.

When a solid body is under the influence of external forces, it naturally deforms. Deformation can be captured by the deformation gradient, which is the derivative of the mapping of a point in the deformed configuration with respect to the position vector in the undeformed configuration. Due to the first and second laws of thermodynamics, the first Piola-Kirchhoff stress tensor is equal to the derivative of a scalar-valued function with respect to the deformation gradient. That scalar-valued function is known as the Helmholtz free-potential density, and it measures the volume density of the strain energy free to do work in each point of the body.

Moving on to finite element analysis. The finite element method assumes the solution to a boundary value problem can be approximated by series of locally simple function. These local approximation is evaluated using finite elements, which are divisions of the body region into smaller partitions. To compute the approximate solution, a linear system of algebraic equations must be assembled and solve. The assembly of this system of equations require the evaluation of the stress tensor in each finite element of the mesh.

In summary, it is essential to have a computational framework to calculate the stress tensor, such as the first Piola-Kirchhoff (PK1) stress tensor, in each finite element. Hence, we must devise a computer program that receives a list of information about each finite element, such as geometric coordinates, and to each region of the domain it belongs. Also, the code must store and use this information to compute the Helmholtz free-potential and, with it, calculate the stress tensor. Finally, this information, which is evaluated element-wise, must

be saved in a single .txt file.

Our proposition is a computer program divided into 4 files—constitutive_model.py (Code 24); numerical_derivative.py (Code 25); stress_calculator.py (Code 26); and main.py (Code 27). The first file will store a class with material parameters and the Helmholtz potential for the Neo-Hookean hyperelastic isotropic model. The second file will store methods to compute the derivative of a generic scalar-valued function of tensor argument using a central finite differences approach. The file stress_calculator.py will retrieve the Helmholtz potential for each class instance and plug it into the second file, in order to evaluate the derivative numerically. The fourth and final file, main.py, will store the user-given data and control the whole process.

This project is optimal to use lists, dictionaries, functions, classes and modularization. We provide the code below and invite the reader to interpret it. All lines are carefully commented to subsidize the reader with enough information to transform the problem described above into a working computer program. We highlight that understanding another's code is an essential exercise to build up one's programming skills and one's programming toolkit.

Before reading the code below, we encourage the reader to try to devise their own logic, then, their own flowchart and logic. After the reader proposes a logic, a pseudo-code and proper python code follow immediately. With all those previous steps complete, the reader should compare their approach with ours.

Is our logic the best option? Is your logic the best option? **Happy coding!**

```
1 # Routine to store a class to evaluate the strain energy density of a
2 # Neo-Hookean isotropic hyperelastic model
3
4 import numpy as np
5
6 class NeoHookean:
7
8     # Defines the initializer function
9
10    def __init__(self, E, nu):
11
12        # Converts the Young modulus (E) and the Poisson ratio (nu) to
13        # the Lamé parameters. Then, stores them in the class
14
15        self.mu = E/(2*(1+nu))
16
17        self.lmbda = nu*E/((1+nu)*(1-2*nu))
18
19    # Defines a method to evaluate the strain energy density using the
20    # deformation gradient as input
21
```

```

22     def energy_density(self, F):
23
24         # Evaluates the right Cauchy-Green strain tensor
25
26         C = np.matmul(F.T, F)
27
28         # Evaluates the trace and the jacobian
29
30         I1 = np.trace(C)
31
32         J = np.sqrt(np.linalg.det(C))
33
34         # Evaluates and returns the strain energy
35
36         ln_J = np.log(J)
37
38         return (0.5*self.mu*(I1-3.0))-(self.mu*ln_J)+((self.lmbda*0.5)*(
39             (ln_J)**2))

```

Code 24: File to store the class for the constitutive model.

```

1 # Routine to store methods to evaluate derivatives numerically using
2 # central finite differences
3
4 import numpy as np
5
6 from copy import deepcopy
7
8 # Defines a function to evaluate the derivative of a scalar function of
9 # tensor argument. The tensor is 3x3
10
11 def derivative_scalar_valued_tensor_argument(f, X, h=1E-5):
12
13     # The derivative of a scalar-valued function of tensor argument with
14     # respect to its argument is another tensor. Thus, initializes the
15     # result of the derivative as a numpy array
16
17     result = np.zeros((3,3))
18
19     # Iterates through the first index
20
21     for i in range(3):
22
23         # Iterates through the second index
24
25         for j in range(3):
26
27             # The X_ij component of X will be perturbed ahead. A copy

```

```

28         # of X must be created to avoid floating point arithmetic
29         # inconsistencies or memory issues
30
31         X_ahead = deepcopy(X)
32
33         X_ahead[i,j] += h
34
35         # Does the same for the abaft perturbation
36
37         X_abaft = deepcopy(X)
38
39         X_abaft[i,j] -= h
40
41         # Gets the quotient and updates the derivative
42
43         result[i,j] = ((f(X_ahead)-f(X_abaft))/(2.0*h))
44
45     # Returns the result
46
47     return result
48
49 # Test block
50
51 if __name__=="__main__":
52
53     # Creates a simple function whose analytical derivative is known.
54     # Let f be the trace function, whose derivative is the identity ten-
55     # sor
56
57     f = lambda F: F[0,0]+F[1,1]+F[2,2]
58
59     df = derivative_scalar_valued_tensor_argument(f, np.array([[1.0,
60     2.0, 3.0], [4.0, 5.0, 6.0], [7.0, 8.0, 9.0]]))
61
62     print("The result of the numerical derivative of the trace is:\n"+
63     str(df))

```

Code 25: File to store functions to evaluate derivatives numerically. In particular, the derivative of a scalar-valued function of tensor argument with respect to its argument. Note the inclusion of a test block as in Appendix B.

```

1 # Routine to store methods to evaluate the stress tensor given a list of
2 # finite elements. The stress tensor is the first Piola-Kirchhoff, and
3 # it is calculated using the derivative of the strain energy density with
4 # respect to the deformation gradient tensor
5
6 import numpy as np
7

```

```

8 from numerical_derivative import derivative_scalar_valued_tensor_argument
9
10 # Defines a function to iterate through the list of finite elements and
11 # get the first Piola-Kirchhoff stress tensor
12
13 def get_first_piola_kirchhoff(finite_elements_info,
14 constitutive_dictionary, deformation_gradient_field):
15
16     # Measures the number of finite elements using the number of sublists
17     # in the list of information of each finite element. This number is,
18     # then, used to initialize the resulting numpy array of stress ten-
19     # sors. This array will have also 9 columns, one for each component
20     # of the first Piola-Kirchhoff (PK1) stress tensor
21
22     PK1_array = np.zeros((len(finite_elements_info), 9))
23
24     # Iterates through the finite elements
25
26     for i in range(len(finite_elements_info)):
27
28         # Retrieves the centroid coordinates and the name of the region
29         # of the finite element
30
31         centroid_x, centroid_y, centroid_z, region_name = finite_elements_info[i]
32
33         # Evaluates the deformation gradient using the given function
34
35         F = deformation_gradient_field(centroid_x, centroid_y, centroid_z)
36
37         # Retrieves the instance of the constitutive model class using
38         # the region name and the dictionary of constitutive models
39
40         constitutive_instance = constitutive_dictionary[region_name]
41
42         # Uses the numerical derivative to evaluate the PK1. The energy
43         # method of the constitutive class is used as scalar function to
44         # be differentiated with respect to F
45
46         PK1 = derivative_scalar_valued_tensor_argument(
47             constitutive_instance.energy_density, F)
48
49         # Updates the array of first Piola-Kirchhoff stress tensors
50
51         PK1_array[i,0] = PK1[0,0]
52
53         PK1_array[i,1] = PK1[0,1]
54

```

```

55     PK1_array[i,2] = PK1[0,2]
56
57     PK1_array[i,3] = PK1[1,0]
58
59     PK1_array[i,4] = PK1[1,1]
60
61     PK1_array[i,5] = PK1[1,2]
62
63     PK1_array[i,6] = PK1[2,0]
64
65     PK1_array[i,7] = PK1[2,1]
66
67     PK1_array[i,8] = PK1[2,2]
68
69     # Returns the array
70
71     return PK1_array

```

Code 26: File to store functions to evaluate stress tensors for each finite element.

```

1  # Routine to control the process of evaluating the first Piola-Kirchhoff
2  # stress tensor at the centroid of each finite element. The deformation
3  # gradient must be supplied as a function of spatial coordinates, and
4  # different regions in the volume are allowed
5
6  import numpy as np
7
8  import stress_calculator
9
10 from constitutive_model import NeoHookean
11
12 # Defines a function to control the whole process and define data
13
14 def process_stresses():
15
16     # Defines the deformation gradient field function
17
18     deformation_gradient_field = lambda x, y, z: np.array([(10.0*x)+1.0,
19     2.0*y, 0.0], [1.5*x, (15*y)+1.0, 0.0], [0.0, 0.0, 1.0]))
20
21     # Defines a dictionary with region names for keys and the constitu-
22     # tive model instances of their class as values
23
24     constitutive_dictionary = {"nucleus": NeoHookean(2E6, 0.4),
25     "ring": NeoHookean(10E6, 0.4)}
26
27     # Defines the list of lists of finite element information. Each sub-
28     # list corresponds to a finite element. Each sublist has the fol-

```

```

29     # lowing information, respectively: x coordinate of the centroid; y
30     # coordinate of the centroid; z coordinate of the centroid; the name
31     # of the region in the domain to which the finite element belongs
32
33     finite_elements_info = [[1.0, 1.0, 1.0, "nucleus"], [1.0, 2.0, 1.0,
34     "nucleus"], [2.0, 1.0, 1.0, "ring"], [2.0, 2.0, 1.0, "ring"]]
35
36     # Gets the stress tensor evaluated at each finite element
37
38     PK1_array = stress_calculator.get_first_piola_kirchhoff(
39     finite_elements_info, constitutive_dictionary,
40     deformation_gradient_field)
41
42     # Saves the array into a .txt file
43
44     np.savetxt("PK1_array.txt", PK1_array)
45
46 # Block to execute the function to kick-start the whole process
47
48 if __name__=="__main__":
49
50     process_stresses()

```

Code 27: File *main.py* used to control the whole process.

Appendix A. Accessing the terminal and using the REPL

Figure A.1 presents instructions to open a terminal window on VSCode (steps 1 and 2). Then, steps 3 and 4 show how to access and use the read-eval-print-loop environment (REPL).

To exit the REPL environment, type on the terminal

```
1 exit()
```

then, press enter.

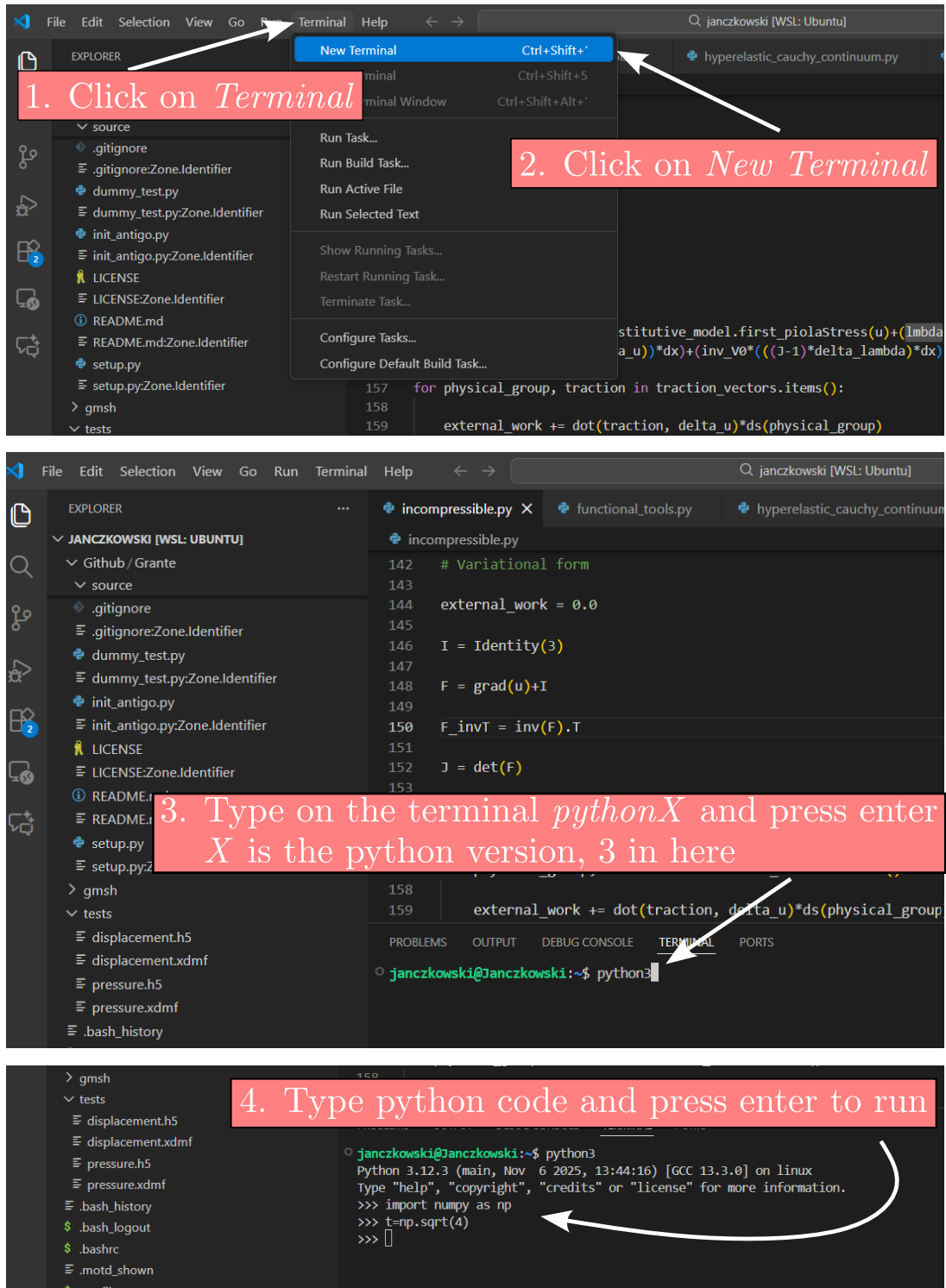


Figure A.1: Instructions to open a terminal and access the REPL in VSCode.

Appendix B. Test block with `__name__ == '__main__'`

Each python module can have a test structure to verify the well-working capability of each one of its functionalities. These tests will be performed only when the module is executed as a file,

```
1 pythonX modulo.py
```

where X is the available python version (example: python3). Otherwise, if the module is executed as a proper module,

```
1 pythonX -m modulo
```

the tests are not run.

It is important to highlight that a module with a built-in test structure **must have all its code** encapsulated in objects such as functions and classes. This way, when the module is loaded as a module, nothing will be automatically executed. Check the example below, and note that the test block is completely defined in the last bit of the code.

```
1 # Generic module with test structure
2
3 import numpy as np
4
5 # Defines a function to calculate the square root of the sum of two numbers
6
7 def sqrt_sum(x, y):
8
9     return np.sqrt(x+y)
10
11 # Defines a test block
12
13 if __name__ == '__main__':
14
15     # Defines known values to test
16
17     x = 10
18
19     y = 6
20
21     # Calls the function to be tested and shows the result
22
23     result = sqrt_soma(x, y)
24
25     print("The result is: "+str(result))
```

Code 28: `__name__ == '__main__'` block.

References

- [1] H. Abelson, G. J. Sussman, J. Sussman, Structure and Interpretation of Computer Programs, 2nd ed., MIT Press/McGraw-Hill, Cambridge, MA, 1996.
- [2] P. Van Roy, S. Haridi, Concepts, Techniques, and Models of Computer Programming, ITPro collection, MIT Press, 2004.
- [3] N. J. Higham, Accuracy and stability of numerical algorithms, 2 ed., Society for Industrial and Applied Mathematics, Philadelphia, Pa., 2002.
- [4] A. Quarteroni, R. Sacco, F. Saleri, Numerical Mathematics, Texts in applied mathematics, 2000 ed., Springer, New York, NY, 2000.
- [5] G. Holzapfel, Nonlinear Solid Mechanics: A Continuum Approach for Engineering, Wiley, 2000.
- [6] J. C. Simo, T. J. R. Hughes, Computational Inelasticity, Interdisciplinary Applied Mathematics, 1 ed., Springer, New York, NY, 2000.
- [7] P. Wriggers, Nonlinear Finite Element Methods, 2008 ed., Springer, Berlin, Germany, 2008.